



**Hochschule
Bonn-Rhein-Sieg**

*University
of Applied Sciences*

Fachbereich Informatik
Department of Computer Sciences

Abschlussarbeit

Studiengang Bachelor Informatik

Entwurf und Evaluation eines fehlertoleranten Job-Processing in Java

von

Max Urfei

Erstprüfer: Prof. Dr. Andreas Hackelöer

Zweitprüfer: M. Sc. Moritz Balg

eingereicht am TT.MM.YYYY

Inhaltsverzeichnis

Abkürzungsverzeichnis	iii
Abbildungsverzeichnis	iv
1 Einleitung	1
1.1 Motivation	1
1.2 Problemstellung	1
1.3 Zielsetzung	2
1.4 Hypothesen	3
1.5 Aufbau der Arbeit	3
2 Grundlagen	4
2.1 Grundlagen der Hintergrundverarbeitung	4
2.1.1 Asynchrone Verarbeitung und Queueing	4
2.1.2 Ausführungssemantiken	6
2.1.3 Background Job-Processing	7
2.2 Konsistenz und konkurrierende Verarbeitung	8
2.2.1 ACID	9
2.2.2 Transaktionsgrenzen	10
2.2.3 Isolationsstufen und Synchronisationsverfahren	11
2.3 Fehlertoleranz und Wiederherstellung	12
2.3.1 Fehlerklassifikation	12
2.3.2 Retry-/Backoff-Strategien	13
2.3.3 Recovery	15
2.3.4 Idempotenz	16
2.4 Stand der Forschung	17
2.4.1 Brokerbasierte Ansätze	17
2.4.2 Datenbankbasierte Ansätze	18
2.4.3 Einordnung und Forschungslücke	18
3 Methodik	20
3.1 Literaturrecherche	20
3.2 Ergebniserzeugung	20
4 Anforderungen	21
4.1 Systemkontext	21
4.2 Abgrenzung des betrachteten Fehlermodells	22
4.3 Zusicherungen	24
4.3.1 Zuverlässige Annahme von Jobs	24
4.3.2 Konsistenz der Verarbeitung	25
4.3.3 Fortsetzung der Verarbeitung	25

5	Umsetzung	27
5.1	Architektur und Design	27
5.1.1	Architekturentwurf	27
5.1.2	Zustandsmodell	28
5.1.3	Entwurf der Datenbank	29
5.1.4	Joberstellung	31
5.1.5	Exklusive Verarbeitung und Transaktionsmodell	32
5.1.6	Retry-Strategie	34
5.1.7	Recovery	36
5.1.8	Idempotente Ergebnisgenerierung	38
5.2	Implementierung	38
5.2.1	Technologie-Stack	39
5.2.2	Zentrale Code-Entscheidungen	39
6	Evaluation	40
6.1	Fehlerszenarien	40
6.2	Durchführung	40
6.3	Ergebnisse	40
7	Diskussion	41
7.1	Gewonnene Erkenntnisse	41
7.2	Einschränkungen und Grenzen der Arbeit	41
8	Fazit und Ausblick	43
	Literaturverzeichnis	44

Abkürzungsverzeichnis

ACID Atomicity, Consistency, Isolation, Durability. 9–11, 18

CRUD Create, Read, Update, Delete. 1

REST Representational State Transfer. 7, 21

UI User Interface. 7, 8

UUID Universally Unique Identifier. 17

Abbildungsverzeichnis

2.1	Vergleich synchroner und asynchroner Kommunikation	5
2.2	Queueing	5
2.3	Verhalten bei transienten Fehlern	14
4.1	Queueing	21
5.1	Abstrakte Architektur des entwickelten Systems	28
5.2	Zustandsmodell eines Jobs	29
5.3	Datenbankschema des entwickelten Job-Processing-Systems	30
5.4	Ablauf der Jobannahme	32
5.5	Transaktionsmodell	33
5.6	Klassendiagramm der Retry-Komponente	36
5.7	Ablauf des Recovery-Mechanismus	37

1 Einleitung

1.1 Motivation

In modernen Softwaresystemen stellt die Verarbeitung von Hintergrundjobs einen zentralen Bestandteil vieler Backend-Architekturen dar (Kleppmann 2017, Kap. 10). Zahlreiche Geschäftsprozesse, darunter die Generierung von Reports, der Import großer Datenmengen, periodische Abrechnungsläufe oder das Versenden asynchroner Benachrichtigungen, erfordern eine Verarbeitung, die entkoppelt vom synchronen Request-Response-Zyklus stattfindet. Insbesondere in industrienahen und sicherheitskritischen Anwendungen, etwa im Bereich der luftfahrttechnischen Wartungsdokumentation, müssen solche Prozesse nicht nur funktional korrekt ablaufen, sondern darüber hinaus zuverlässig, nachvollziehbar und fehlertolerant sein. Aber auch in nicht-kritischen Systemen ist Zuverlässigkeit von Bedeutung für die Nutzerfreundlichkeit der Anwendung, als auch um Produktivitäts- und Gewinneinbußen in Produkktivsystemen in Folge von Fehlern zu vermeiden (Kleppmann 2017, Kap. 1 Abschnitt „How Important Is Reliability?“).

1.2 Problemstellung

In der Praxis können bei der Verarbeitung von Hintergrundjobs jederzeit Teilausfälle auftreten. Prozesse können während der Verarbeitung unerwartet terminieren, Datenbankverbindungen können kurzzeitig unterbrochen werden oder identische Anfragen werden zum Beispiel infolge von clientseitigen Wiederholungsversuchen mehrfach an das System übermittelt oder Ergebnisse bzw. Antworten gehen verloren (Kleppmann 2017, Kap. 8). Ohne geeignete Mechanismen führen derartige Szenarien zu inkonsistenten Systemzuständen. Beispielsweise verbleiben Jobs im Status „laufend“, obwohl kein Worker sie mehr aktiv verarbeitet, Ergebnisse werden doppelt erzeugt oder Aufträge gehen vollständig verloren.

Bestehende Frameworks und Bibliotheken, etwa Quartz Scheduler (Terracotta Inc. (Hrsg.) 2026) oder Spring Batch (Broadcom Inc. (Hrsg.) 2026b), stellen umfangreiche Funktionalitäten für die Hintergrundverarbeitung bereit. Hierzu zählen unter anderem Scheduling, Wiederholungsmechanismen sowie Verfahren zur Fehlerbehandlung und Wiederaufnahme. Sie verfolgen dabei jedoch jeweils eigene Architekturansätze und abstrahieren die zugrunde liegenden Mechanismen weitgehend hinter ihren Programmierschnittstellen. Dadurch eignen sie sich nur eingeschränkt, um systematisch zu untersuchen, welche Architekturentscheidungen für bestimmte Konsistenz- und Verarbeitungszusicherungen tatsächlich erforderlich sind.

Die beschriebene Problematik ist von hoher praktischer und industrieller Relevanz. Nahezu jedes Backend-System, das über einfache Create, Read, Update, Delete (CRUD)-Operationen hinausgeht, muss Aufgaben zuverlässig im Hintergrund verarbeiten und dabei konsistente Datenbestände gewährleisten (Richardson 2018, Kap. 4). Fehler in diesem Bereich führen zu Datenverlust, inkonsistenten Geschäftszuständen sowie unnöti-

gem manuellem Eingriff und können insbesondere in regulierten Domänen wie der Luftfahrt Compliance-Verstöße nach sich ziehen. Auch im Kontext von Cloud-nativen Architekturen ist dieses Thema relevant. Horizontale Skalierung, Container-Neustarts und kurzzeitige Netzwerkunterbrechungen gehören dort zum regulären Betriebsverhalten und müssen bereits beim Entwurf eines Systems berücksichtigt werden. Daher bilden die in dieser Arbeit behandelten Konzepte auch die Grundlage für den zuverlässigen Betrieb von Backend-Systemen in Cloud-Umgebungen.

Obwohl die zugrundeliegenden Konzepte wie Fehlertoleranz, Recovery, Idempotenz, Nebenläufigkeitskontrolle und Transaktionsmanagement in der Literatur umfassend beschrieben werden, existieren nur wenige Arbeiten, die diese Aspekte zu einer durchgängigen datenbankbasierten Architektur für Background-Job-Processing zusammenführen und deren Verhalten unter definierten Fehlerszenarien systematisch evaluieren. Ebenso implementieren bestehende Frameworks diese Mechanismen häufig als interne Abstraktionen, wodurch deren Zusammenspiel und Auswirkungen auf die Zuverlässigkeit des Gesamtsystems für Entwickler nur eingeschränkt nachvollziehbar sind.

Daraus ergibt sich die dieser Arbeit zugrundeliegende Hauptforschungsfrage:
Welche Architekturmechanismen und Entwurfsentscheidungen sind notwendig, um in einem Java/Spring-basierten Job-Processing-Backend definierte Konsistenzzusicherungen unter typischen Teilausfallszenarien nachweisbar einzuhalten?

Zur Beantwortung dieser Frage werden folgende Teilfragen untersucht:

- TF1: Welche Zusicherungen muss das System gewährleisten, damit Jobs stets korrekt und ohne Inkonsistenzen verarbeitet werden?
- TF2: Welchen Beitrag leisten Idempotenz, Retry-Strategien und Recovery-Mechanismen zur Robustheit unter definierten Fehlerszenarien?
- TF3: Wie müssen Transaktionen und Locking-Strategien genutzt werden, damit bei paralleler Arbeit mehrerer Worker keine Doppelverarbeitung oder inkonsistenten Zustände entstehen?
- TF4: Wie lässt sich sicherstellen, dass Jobs nach einem unerwarteten Prozessabbruch korrekt wiederaufgenommen werden, ohne dass Aufträge verloren gehen?

1.3 Zielsetzung

Ziel dieser Arbeit ist der Entwurf, die prototypische Implementierung und die Evaluation eines fehlertoleranten Job-Processing-Backends auf Basis einer relationalen Datenbank in Kombination mit Java und Spring. Dabei soll herausgearbeitet werden, welche Architekturmechanismen erforderlich sind, damit Hintergrundjobs auch bei Teilausfällen zuverlässig, korrekt und nachvollziehbar verarbeitet werden und definierte Konsistenzzusicherungen eingehalten werden.

Konkret verfolgt die Arbeit folgende Teilziele:

- TZ1: Definition von Zusicherungen: Es sollen präzise Korrektheitsbedingungen formuliert werden, die das System zu jedem Zeitpunkt einhalten muss, unabhängig davon, welche Fehler auftreten. Dazu zählen unter anderem, dass kein Job doppelt erfolgreich abgeschlossen wird, dass kein Job dauerhaft verloren geht und dass Statusübergänge ausschließlich dem definierten Zustandsmodell folgen.
- TZ2: Architekturentwurf: Es soll eine Architektur entworfen werden, die persistente Job-States, Idempotenz, Retry-Strategien sowie ein nachvollziehbares Statusmodell umfasst. Architekturentscheidungen sollen explizit dokumentiert und begründet werden.
- TZ3: Prototypische Implementierung: Die entworfene Architektur soll als funktionsfähiger Prototyp umgesetzt werden, der die definierten Zusicherungen unter realistischen Bedingungen einhält.
- TZ4: Evaluation unter Fehlerbedingungen: Es soll nachgewiesen werden, dass das System seine Zusicherungen auch bei gezielt herbeigeführten Teilausfällen nicht verletzt.

1.4 Hypothesen

Auf Basis der Forschungsfrage und der Zielsetzung werden folgende Hypothesen formuliert, deren Überprüfung im Rahmen der Implementierung und insbesondere der Evaluation erfolgt.

- H1: Durch die Kombination eines persistenten Statusmodells mit transaktionsbasiertem Locking und Idempotenz-Mechanismen lässt sich ein System entwerfen, das auch bei unerwarteten Prozessabbrüchen keine Jobs verliert und keine doppelten Ergebnisse erzeugt.
- H2: Retry-Strategien mit exponentiellem Backoff in Verbindung mit einer Fehlerklassifikation (transient vs. permanent) erhöhen die Robustheit des Systems gegenüber kurzzeitigen Infrastrukturausfällen, ohne dabei die definierten Zusicherungen zu verletzen.
- H3: Die definierten Zusicherungen lassen sich durch automatisierte Tests mit gezielter Fehlerinjektion reproduzierbar überprüfen und nachweisen.

1.5 Aufbau der Arbeit

2 Grundlagen

Dieses Kapitel vermittelt die theoretischen Grundlagen, die für das Verständnis der im weiteren Verlauf entwickelten Architektur erforderlich sind. Zunächst werden die grundlegenden Konzepte der Hintergrundverarbeitung sowie die Anforderungen an zuverlässige Background-Job-Processing-Systeme erläutert. Darauf aufbauend werden die für diese Arbeit relevanten Mechanismen zur Gewährleistung von Konsistenz bei paralleler Verarbeitung sowie Konzepte zur Fehlertoleranz, Wiederherstellung und idempotenten Verarbeitung eingeführt. Die dargestellten Grundlagen bilden die Basis für den in Kapitel 5.1 vorgestellten Architekturentwurf und dessen anschließende Implementierung und Evaluation.

2.1 Grundlagen der Hintergrundverarbeitung

Hintergrundverarbeitung dient dazu, zeitintensive oder ressourcenintensive Aufgaben vom synchronen Request-Response-Zyklus zu entkoppeln. Gleichzeitig entstehen daraus jedoch auch einige Herausforderungen: In verteilten Umgebungen sind Teilausfälle, Timeouts und kurzzeitige Unterbrechungen üblich, sodass ohne entsprechende Mechanismen nicht davon ausgegangen werden kann, dass eine angestoßene Verarbeitung vollständig und genau einmal ausgeführt wird. Insbesondere kann bei Kommunikationsabbrüchen oder Wiederholungsversuchen unklar bleiben, ob eine Operation bereits verarbeitet wurde oder ob sie erneut angestoßen werden muss (Kleppmann 2017, Kap. 8). Vor diesem Hintergrund werden in den folgenden Abschnitten grundlegende Konzepte vorgestellt, die eine Entkopplung der Verarbeitung ermöglichen und die Basis für robuste Hintergrundverarbeitung bilden.

2.1.1 Asynchrone Verarbeitung und Queueing

In Backend-Systemen lässt sich Kommunikation und Verarbeitung bezüglich der zeitlichen Kopplung zwischen synchronem und asynchronem Verhalten unterscheiden. Der Unterschied der Funktionsweise wird in Abbildung 2.1 deutlich. Bei synchroner Verarbeitung nach dem Request-Response-Stil wartet der Aufrufer auf eine Antwort auf die Anfrage und blockiert gegebenenfalls bevor er fortfährt. Bei asynchroner Verarbeitung wird die Antwort auf den Auftrag vom Zeitpunkt des Erhalts entkoppelt. Der Aufrufer initiiert die Arbeit, blockiert jedoch nicht und kann auch während des Wartens auf eine Antwort weiter arbeiten (Richardson 2018, Kap. 3.1.1).

Zur praktischen Umsetzung asynchroner Verarbeitung werden häufig Puffer eingesetzt. Dieses Prinzip wird als Queueing bezeichnet. Dadurch können Nachrichtenverluste reduziert und Lastspitzen ausgeglichen werden, indem der Puffer erhaltene Anfragen zur späteren Ausführung zwischenspeichert (Kleppmann 2017, Kap. 11, Abschnitt „Message brokers“). Das grundlegende Prinzip ist in Abbildung 2.2 schemenhaft dargestellt. Ein solcher Puffer kann verschiedene Ausprägungen annehmen, beispielsweise als In-Memory beim Empfänger, als Datenbank oder in Form eines Message Brokers. Ein Produzent

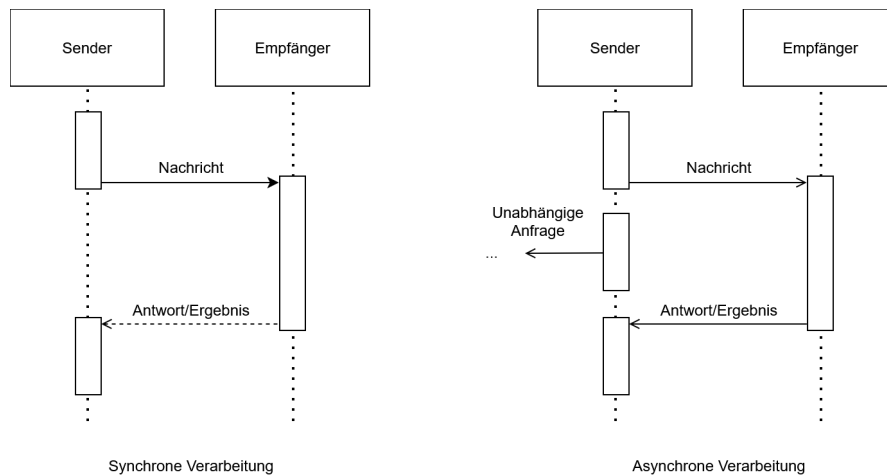


Abbildung 2.1: Vergleich synchroner und asynchroner Kommunikation

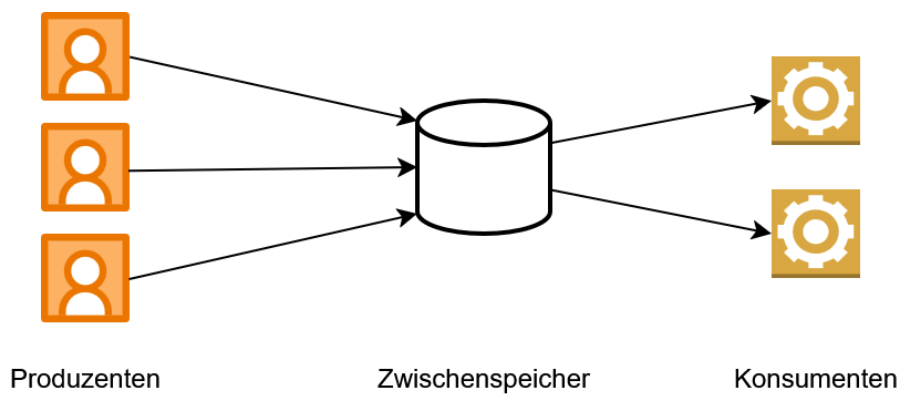


Abbildung 2.2: Queueing

sendet eine Nachricht an einen Zwischenspeicher und wartet in der Regel lediglich auf das Acknowledgment, dass die Nachricht zwischengespeichert wurde, nicht jedoch auf die Verarbeitung oder das Ergebnis (Kleppmann 2017, Kap. 11, Abschnitt „Messaging Systems“). Die so erreichte Entkopplung hat gleichzeitig den Vorteil, dass die Verarbeitung der Anfrage unabhängig von der Verbindung zwischen Produzent und Konsument wird. Im Gegensatz dazu kann bei synchronen Systemen bei Verbindungsabbrüchen oder Timeouts unklar bleiben, ob eine Anfrage den Server erreicht hat oder bereits verarbeitet wurde. Genau dieses Problem adressieren Message Broker: Falls ein Konsument temporär nicht verfügbar ist oder während der Verarbeitung ausfällt, können Message Broker trotzdem für die At-Least-Once-Semantik (vgl. Kapitel 2.1.2) sorgen (Kleppmann 2017, Kap. 11, Abschnitt „Acknowledgments and redelivery“). Des Weiteren müssen Verarbeitungskapazitäten in Folge von Queueing nicht auf die maximal zu erwartende Last ausgelegt werden, sondern können sich an der durchschnittlichen Auslastung orientieren.

Kurzzeitige Lastspitzen werden dabei durch die Zwischenspeicherung der Aufgaben in der Queue ausgeglichen (Microsoft Corporation (Hrsg.) 2026b). Die synchrone Request-Response-Kommunikation bietet demgegenüber den Vorteil einer unmittelbaren Rückmeldung über Erfolg oder Fehler einer Operation, setzt jedoch voraus, dass Client und Server während der gesamten Interaktion verfügbar sind (Richardson 2018, Kap. 3.4).

Die asynchrone Verarbeitung bildet damit die Grundlage für die Hintergrundverarbeitung, indem die Entkopplung genutzt wird, um Aufgaben unabhängig vom ursprünglichen Aufruf auszuführen und gleichzeitig die Robustheit und Skalierbarkeit des Systems zu erhöhen (Newman 2026, Kap. 8, Abschnitt „Why Use A Message Broker?“).

2.1.2 Ausführungssemantiken

In verteilten Systemen führen potenzielle Prozessabstürze und Teilausfälle dazu, dass Aufträge verloren gehen oder der Verarbeitungszustand einer Aufgabe unklar sein kann. Um zu beschreiben, was Systeme aufgrund ihres Entwurfs und ihrer Mechanismen zur Fehlertoleranz bezüglich Auslieferung beziehungsweise Ausführung garantieren, werden grundsätzlich drei fundamentale Ausführungssemantiken unterschieden, die Newman (2026, Kap. 8 Abschnitt „Types of Delivery Guarantees“) vor dem Hintergrund der Auslieferung wie folgt definiert. Im Kontext von Job-Processing sind diese Semantiken zur Nachrichtenzustellung analog auf die Ausführung der Jobs anwendbar.

At-Least-Once

Ein Konsument erhält die Nachricht mindestens einmal. Bei der Auslieferung bestätigt der Konsument über ein Acknowledge das Ankommen der Nachricht. Sofern das Acknowledgement beim Absender eingeht, ist für diesen garantiert, dass die Nachricht angekommen ist. Bleibt diese Bestätigung innerhalb eines definierten Zeitfensters jedoch aus, z.B. weil die Nachricht oder das Acknowledgement verloren geht, wird die Nachricht erneut zugestellt. Dieser Fall wird Redelivery genannt. Dieses Verfahren verhindert Datenverlust, birgt jedoch das Risiko von Duplikaten, für den Fall, dass die Nachricht ankommt, aber das Acknowledge verloren geht und es infolgedessen zur Redelivery kommt. Einschränkung sei hier gesagt, dass die Zustellung nicht garantiert werden kann, wenn der Konsument bei keinem Zustellversuch verfügbar ist.

At-Most-Once

Ein Konsument erhält die Nachricht maximal einmal. Der Produzent schickt die Nachricht ab und erwartet keine Empfangsbestätigung. Eine erneute Zustellung findet unter keinen Umständen statt. Dadurch werden Duplikate strikt ausgeschlossen, jedoch muss ein potenzieller Datenverlust zugunsten besserer Latenzen hingenommen werden.

Exactly-Once

Diese Semantik garantiert, dass eine Nachricht genau einmal beim Konsumenten ankommt. In der Praxis ist das nur schwer zu realisieren, da beispielsweise nicht klar sein

kann, ob ein Acknowledgement nur noch nicht angekommen, oder schon verloren gegangen ist. Viele Umsetzungen beruhen daher auf der Notwendigkeit, dass sowohl beim Produzent, als auch Konsument entsprechende Mechanismen implementiert werden. Eine Variante zur Umsetzung von Exactly-Once ist die Approximation über eine Kombination der Implementierung von At-Least-Once beim Produzent und Idempotenz beim Konsument. Die Garantie bezieht sich somit nicht auf die physische Zustellung einer Nachricht, sondern auf die Wirkung ihrer Verarbeitung. Eine Nachricht kann mehrfach zugestellt werden, solange ihre Verarbeitung nur einmal wirksam wird.

2.1.3 Background Job-Processing

Aufbauend auf den Konzepten der asynchronen Verarbeitung (vgl. Kapitel 2.1.1) bezeichnet das Background Job-Processing die Ausführung von Aufgaben außerhalb des Request-Response-Zyklus. Hierbei werden fachliche Operationen als eigenständige Arbeitseinheiten (Jobs) modelliert und zeitlich entkoppelt verarbeitet.

In der Softwarearchitektur existieren verschiedene etablierte Entwurfsmuster zur Umsetzung einer solchen Infrastruktur, wie beispielsweise das Queue-Based Load Leveling Pattern (Microsoft Corporation (Hrsg.) 2026b). Dieses Muster besteht ähnlich des Konzepts aus Abbildung 2.2 aus drei funktionalen Kernkomponenten:

- **Der Produzent:** Erzeugt einen Job und gibt diesen weiter an den Job-Store. Das kann zum einen direkt durch das User Interface (UI) geschehen, oder die UI ruft einen Endpoint auf, der den Job über eine synchrone Schnittstelle, wie z.B. Representational State Transfer (REST), entgegen nimmt. In letzterem Fall fungiert der Service hinter dem Endpoint als Produzent. Dessen Aufgabe besteht darin, den Auftrag zu validieren und an die Persistenzschicht zu übergeben, woraufhin der synchrone Request-Response-Zyklus für den Client sofort beendet wird. Jobs können aber nicht nur durch solche Event-driven Trigger ausgelöst werden, sondern auch durch Schedule-driven Trigger. Dabei erfolgt die Ausführung eines Jobs periodisch oder zu vordefinierten Zeitpunkten, z.B. über durch einen Cron-Trigger (Microsoft Corporation (Hrsg.) 2026a).
- **Der Job-Store:** Speichert die Jobs für den Zeitraum von der Erstellung bis zur endgültigen Beendigung zwischen. Für die Umsetzung existieren verschiedene Varianten, beispielsweise als Message Broker oder relationale Datenbank. Diese Realisierungsmöglichkeiten bieten jeweils spezifische Vor- und Nachteile in der Nutzung. Die Auswahl des passenden Ansatzes sollte daher durch die Qualitätsanforderungen an das System, wie z.B. Durchsatz oder Akzeptanz einzelner verlorener Jobs, begründet werden (Kleppmann 2017, Kap. 11).
- **Der Worker:** Autonome Komponente, die als Konsument fungiert. Sobald der Worker über freie Systemressourcen verfügt, kann er einen wartenden Job aus dem Job-Store beanspruchen und führt diesen aus (Microsoft Corporation (Hrsg.) 2026b).

Durch eine solche Architektur können Background Jobs als „fire and forget“-Operationen (Microsoft Corporation (Hrsg.) 2026a) implementiert werden. Sie laufen asynchron in einem vom Client isolierten Prozess, sodass deren Verarbeitungszeit keinerlei unmittelbare Auswirkung auf die Reaktionszeit oder Verfügbarkeit der UI hat (Microsoft Corporation (Hrsg.) 2026a). Außerdem ermöglicht dieses Konzept das Abfangen von Lastspitzen und flexiblere Skalierung (Microsoft Corporation (Hrsg.) 2026b).

Die Zuweisung von Jobs an Worker kann grundsätzlich über unterschiedliche Modelle erfolgen. Beim Push-Modell weist eine zentrale Komponente Aufgaben aktiv freien Worker-Instanzen zu. Beim Pull-Modell fragen Worker eigenständig neue Aufgaben aus dem Job-Store ab. Beide Ansätze unterscheiden sich hinsichtlich Komplexität, Skalierbarkeit und Koordinationsaufwand (Kleppmann 2017, Kap. 11 Abschnitt „Transmitting Event Streams“).

Werden mehrere Worker parallel betrieben, entsteht die Herausforderung der konkurrierenden Verarbeitung. Ohne geeignete Koordinationsmechanismen könnten mehrere Instanzen denselben Job gleichzeitig beanspruchen oder widersprüchliche Statusänderungen durchführen. Die hierfür relevanten Konzepte werden in Kapitel 2.2 behandelt.

Da die Verarbeitung zeitlich entkoppelt erfolgt, müssen Informationen über offene und bereits bearbeitete Jobs über den Lebenszyklus einzelner Prozesse hinweg erhalten bleiben. Hierfür ist eine persistente Speicherung des Verarbeitungszustands erforderlich. Insbesondere in verteilten Systemen können Teilausfälle auftreten, bei denen einzelne Komponenten oder Prozesse während der Ausführung ausfallen, während andere Teile des Systems weiterhin verfügbar bleiben. Damit eine unterbrochene Verarbeitung nach einem solchen Ausfall wieder aufgenommen werden kann, muss das System den zuletzt bekannten Zustand kennen oder anhand persistenter Informationen rekonstruieren können (Kleppmann 2017, Kap. 11, Abschnitt „Rebuilding state after a failure“).

Neben Prozessabstürzen können auch externe Dienste vorübergehend nicht erreichbar sein oder Kommunikationsfehler auftreten. Das System muss daher abhängig von der jeweiligen Fehlersituation entscheiden können, ob eine Verarbeitung erneut durchgeführt, fortgesetzt oder endgültig beendet werden soll (Microsoft Corporation (Hrsg.) 2026a). Die hierfür relevanten Konzepte zur Fehlerbehandlung und Wiederherstellung werden in Kapitel 2.3 vorgestellt.

Da sowohl Wiederholungsversuche als auch Wiederaufnahmen nach einem Ausfall dazu führen können, dass einzelne Verarbeitungsschritte mehrfach ausgeführt werden, muss darüber hinaus sichergestellt werden, dass solche Mehrfachausführungen keine inkonsistenten Ergebnisse verursachen. Hierzu werden Verfahren der idempotenten Verarbeitung eingesetzt, die in Kapitel 2.3.4 vorgestellt werden.

2.2 Konsistenz und konkurrierende Verarbeitung

Bei der Verarbeitung von Daten durch mehrere parallel arbeitende Prozesse entsteht die Herausforderung, dass konkurrierende Zugriffe auf gemeinsame Ressourcen zu inkonsistenten Zuständen führen können. Ohne geeignete Mechanismen können Änderungen verloren gehen, widersprüchliche Zustände entstehen oder mehrere Prozesse dieselben

Daten gleichzeitig verändern (Saake, Heuer und Sattler 2018, Kap. 13.1.2).

Im Kontext von Background Job-Processing tritt dieses Problem besonders dann auf, wenn mehrere Worker-Instanzen parallel auf denselben Jobbestand zugreifen. Versuchen sie ohne geeignete Synchronisationsmechanismen denselben Job zu beanspruchen oder konkurrierende Statusänderungen durchzuführen, können Race Conditions entstehen. Insbesondere bei Pull-basierten Architekturen, bei denen die Jobverteilung nicht durch eine Orchestrierungskomponente realisiert wird, muss eine solche ungewollte Doppelbeanspruchung explizit verhindert werden (Kleppmann 2017, Kap. 7 Abschnitt „Isolation“). Die Gewährleistung konsistenter Zustände stellt daher eine zentrale Anforderung an Systeme mit paralleler Verarbeitung dar. Die hierfür etablierten Konzepte und Mechanismen werden in den folgenden Abschnitten vorgestellt.

2.2.1 ACID

Zur Vereinfachung der Gewährleistung konsistenter Zustände bei parallelen Zugriffen existiert das Konzept der Transaktionen. Diese fassen mehrere Operationen zu einer logischen Einheit zusammen, deren Ausführung nach außen als zusammenhängender Vorgang erscheint (Kleppmann 2017, Kap. 7). Im Kontext persistenter Datenhaltung werden von Transaktionen typischerweise die vier zentralen Eigenschaften Atomicity, Consistency, Isolation, Durability (ACID) gefordert (Saake, Heuer und Sattler 2018, Kap. 13.1.1):

Atomicity Die Atomarität stellt sicher, dass eine Transaktion entweder vollständig oder gar nicht ausgeführt wird. Tritt während der Ausführung ein Fehler auf, werden sämtliche bis dahin vorgenommenen Änderungen verworfen, sodass keine partiellen Zwischenzustände bestehen bleiben.

Consistency Die Konsistenz beschreibt die Eigenschaft, dass eine erfolgreich abgeschlossene Transaktion die Daten von einem gültigen Zustand in einen anderen gültigen Zustand überführt. Dabei müssen definierte Integritätsbedingungen erhalten bleiben. Kleppmann (2017, Kap. 7 Abschnitt „Consistency“) weist darauf hin, dass die Einhaltung solcher fachlichen Invarianten in der Verantwortung der Anwendung liegt und nicht wie die anderen Eigenschaften durch das Datenbanksystem garantiert werden kann.

Isolation Die Isolation gewährleistet, dass parallel ausgeführte Transaktionen sich nicht gegenseitig beeinflussen. Aus Sicht einer Transaktion soll die Ausführung so erscheinen, als würde sie allein auf dem Datenbestand arbeiten. Dadurch können Race Conditions und andere Nebenläufigkeitsprobleme vermieden werden.

Durability Die Dauerhaftigkeit garantiert, dass die Ergebnisse einer erfolgreich abgeschlossenen Transaktion auch nach Systemabstürzen oder anderen Fehlern dauerhaft erhalten bleiben.

Im Kontext von Background Job-Processing sind insbesondere die Eigenschaften der Atomarität und Isolation von Bedeutung. Während die Atomarität verhindert, dass Statusänderungen eines Jobs nur teilweise durchgeführt werden, bildet die Isolation die

Grundlage für die koordinierte Verarbeitung gemeinsamer Datenbestände durch mehrere parallel arbeitende Worker-Instanzen. Die relevanten Konzepte zum Erreichen dieser Eigenschaften werden in den folgenden Abschnitten näher betrachtet.

2.2.2 Transaktionsgrenzen

Während ACID die theoretischen Garantien einer Transaktion definiert, bestimmt die Festlegung von Transaktionsgrenzen, welche konkreten Datenbankoperationen des softwareseitigen Kontrollflusses zu einer unteilbaren, logischen Einheit zusammengefasst werden. Die Festlegung dieser Grenzen steuert, wann eine Datenbanksitzung eine Transaktion initiiert und zu welchem Zeitpunkt die Änderungen final festgeschrieben (Commit) oder im Fehlerfall vollständig verworfen (Rollback) werden (Kudraß 2015, S.250).

Die Wahl geeigneter Transaktionsgrenzen wird zusätzlich dadurch erschwert, dass fachliche und technische Transaktionen nicht zwangsläufig identisch sind. Eine fachliche Transaktion beschreibt eine zusammenhängende Geschäftsoperation, deren Ergebnis aus Sicht der Anwendung als Einheit betrachtet wird. Eine technische Transaktion hingegen bezeichnet die konkrete Transaktion auf der Datenbankebene, welche die ACID-Eigenschaften für die von ihr gekapselten Datenbankoperationen gewährleistet. Während eine fachliche Operation mehrere technische Transaktionen umfassen kann, können innerhalb einer technischen Transaktion auch nur Teilaspekte einer fachlichen Operation verarbeitet werden. Werden die Grenzen falsch gesetzt, können verschiedene Parallelitätsanomalien auftreten (Saake, Heuer und Sattler 2018, Kap. 13.1.2). Ein verbreiteter Ansatz zur Vermeidung dieser Anomalien ist die Verwendung von Sperren, auf die in Kapitel 2.2.3 näher eingegangen wird. Wichtig ist an dieser Stelle, dass die gewählten Transaktionsgrenzen direkten Einfluss auf die Dauer von Sperren und somit auch auf parallel ausgeführte Transaktionen haben.

Eine Möglichkeit besteht darin, jede Datenbankoperation in einer eigenen Transaktion auszuführen. Dadurch werden Sperren nur für sehr kurze Zeit gehalten, was die Parallelität und den Durchsatz des Systems erhöht. Allerdings können zusammengehörige Änderungen nicht mehr atomar ausgeführt werden. Tritt zwischen mehreren aufeinanderfolgenden Operationen ein Fehler auf, können bereits festgeschriebene Änderungen nicht mehr gemeinsam zurückgesetzt werden. Die betroffenen Daten können dadurch in einen fachlich falschen Zustand überführt werden.

Als naiver Gegenentwurf können die Transaktionsgrenzen entsprechend der fachlichen Transaktion gesetzt werden. Diese Strategie kann jedoch zu langlaufenden Transaktionen führen, welche die Leistungsfähigkeit des Datenspeichers beeinträchtigt. Zum einen halten Transaktionen im Beispiel des 2-Phasen-Sperrprotokolls sämtliche während ihrer Laufzeit akquirierten Sperren bis zum endgültigen Commit oder Rollback aufrecht (Kleppmann 2017, Kap. 7 Abschnitt „Two-Phase Locking (2PL)“). Dadurch steigt die Wahrscheinlichkeit, dass konkurrierende Transaktionen aufeinander warten müssen. Zum anderen können langlaufende Transaktionen die Bereinigung alter Datenversionen verzögern, da diese weiterhin für aktive Transaktionen sichtbar bleiben müssen (The PostgreSQL Global Development Group (Hrsg.) 2026, Kap. 24.1.2). Dies erhöht den Ressourcenbedarf des Datenbanksystems und kann dessen Leistungsfähigkeit beeinträchtigen.

Im Software-Entwurf erfordert das Setzen der Transaktionsgrenzen daher eine kontinuierliche Abwägung zwischen der Systemperformance und dem Schutz vor Datenanomalien. Weit gefasste Transaktionsgrenzen reduzieren das Risiko der Entstehung von Inkonsistenzen, blockieren jedoch Systemressourcen und Datensätze. Eng gefasste Grenzen erhöhen den Durchsatz, können jedoch das Risiko konkurrierender Zugriffe und daraus resultierender Race Conditions erhöhen.

2.2.3 Isolationsstufen und Synchronisationsverfahren

Zur Umsetzung der Isolationseigenschaft von Transaktionen müssen konkurrierende Zugriffe auf gemeinsame Daten koordiniert werden. Zu diesem Zweck stellen Datenbanksysteme unterschiedliche Isolationsstufen bereit. Diese schwächen Teile der ACID-Garantien bewusst ab und legen damit fest, in welchem Umfang Transaktionen die Änderungen parallel laufender Transaktionen beobachten und welche Parallelitätsanomalien dadurch im System auftreten können (Kudraß 2015, S. 251). Je nach Anwendungskontext kann es sein, dass nicht alle dieser Anomalien auftreten können, beispielsweise wenn nur Leseoperationen ausgeführt werden, oder dass deren Auftreten in Kauf genommen werden kann, um eine bessere Performance zu erhalten. Dadurch müssen weniger Überprüfungen stattfinden und weniger Transaktionen müssen zurückgesetzt werden, was zu einer besseren Performance führt. Höhere Isolationsstufen bieten stärkere Konsistenzgarantien, können jedoch die Parallelität des Systems verringern (Saake, Heuer und Sattler 2018, Kap. 13.1.3). Während die Isolationsstufen somit die Konsistenzgarantien definieren, kommen zur Koordination der Transaktionen im Datenbanksystem verschiedene Synchronisationsverfahren zum Einsatz, die durch sperrende oder nicht-sperrende Strategien das Entstehen nicht zugelassener Anomalien verhindern (Kudraß 2015, S. 253–259).

Eine verbreitete Strategie zur Durchsetzung der Isolation ist die pessimistische Synchronisation. Sie basiert auf der Annahme, dass konkurrierende Zugriffe auftreten können und verhindert diese bereits im Vorfeld durch Sperrmechanismen. Bevor eine Transaktion einen Datensatz verändert, wird dieser für andere konkurrierende Transaktionen gesperrt. Der Vorteil pessimistischer Verfahren besteht darin, dass Konflikte bereits vor ihrer Entstehung verhindert werden. Dadurch eignen sie sich insbesondere für Szenarien mit häufigen konkurrierenden Zugriffen. Nachteilig wirken sich jedoch zusätzliche Wartezeiten aus, da Transaktionen auf die Freigabe benötigter Sperren warten müssen. Bei hoher Auslastung kann dies die Parallelität und den Gesamtdurchsatz eines Systems reduzieren. Außerdem entsteht die Gefahr von Deadlocks (Kudraß 2015, S. 253–257).

Alternative Ansätze sind nicht sperrende Verfahren, beispielsweise die optimistische Synchronisation oder das Zeitstempelverfahren. Anstatt Daten vor der Bearbeitung zu sperren, werden bei der optimistischen Synchronisation Transaktionen zunächst ausgeführt und erst beim Schreiben geprüft, ob ein Konflikt entstanden ist. Ist es zwischen zwei oder mehreren Transaktionen zum Konflikt gekommen, müssen einzelne Transaktionen zurückgesetzt und wiederholt werden. Bei dem Zeitstempelverfahren werden Zeitstempel für jede Transaktionen, sowie der Zeitpunkt des letzten Lese- und Schreibzugriffs für jedes Datenbankobjekt verwaltet. Anhand dieser Zeitstempel kann geprüft werden, ob zwei

Transaktionen in Konflikt zueinander stehen und ob eine der Transaktionen zurückgesetzt werden muss. Nicht sperrende Verfahren vermeiden die durch Sperren verursachten Wartezeiten und ermöglichen dadurch eine hohe Parallelität. Treten jedoch Konflikte auf, müssen betroffene Operationen wiederholt werden. Sie eignen sich daher insbesondere für Systeme, in denen konkurrierende Änderungen selten vorkommen (Kudraß 2015, S. 257–259).

Welche Synchronisationsstrategie geeignet ist, hängt von den Anforderungen des jeweiligen Systems und der Häufigkeit erwarteter Konflikte ab. Während pessimistische Verfahren Konflikte präventiv verhindern, akzeptieren optimistische Verfahren mögliche Konflikte und behandeln diese nachträglich.

2.3 Fehlertoleranz und Wiederherstellung

Verteilte Softwaresysteme sind zwangsläufig mit einer Vielzahl potenzieller Fehlerquellen konfrontiert. Prozesse können unerwartet terminieren, Netzwerkverbindungen unterbrochen werden oder externe Dienste zeitweise nicht erreichbar sein (Newman 2026, Kap. 1 Abschnitt „The Three Golden Rules Of Distributed Systems“). Beispielsweise kann es bei einzelnen Teilausfällen dazu kommen, dass unklar bleibt, ob ein Verarbeitungsschritt erfolgreich abgeschlossen wurde oder erneut ausgeführt werden muss. Da sich solche Fehler in komplexen Systemlandschaften nicht vollständig vermeiden lassen, muss ihre Existenz als grundlegende Systemeigenschaft betrachtet werden (Newman 2026, Kap. 1 Abschnitt „Failure Is Inevitable“).

Die Zuverlässigkeit eines Systems ergibt sich daher nicht primär daraus, Fehler vollständig zu verhindern, sondern daraus, wie gut mit ihrem Auftreten umgegangen wird. Fehlertoleranz bezeichnet in diesem Zusammenhang die Fähigkeit eines Systems, seine wesentlichen Funktionen trotz auftretender Störungen weiterhin bereitzustellen. Darüber hinaus muss ein System in der Lage sein, nach einem Fehler einen konsistenten Zustand wiederherzustellen und die Verarbeitung kontrolliert fortzusetzen, beziehungsweise neuzustarten (Newman 2026, Kap. 1 „What Is Resilience?“). Die folgenden Abschnitte stellen die hierfür relevanten Konzepte und Mechanismen vor.

2.3.1 Fehlerklassifikation

Da Fehler in verteilten Systemen unvermeidbar sind, stellt sich nicht die Frage, ob sie auftreten, sondern wie mit ihnen umgegangen werden soll. Die Wirksamkeit von Fehlertoleranzmechanismen hängt dabei wesentlich davon ab, die Ursache und Eigenschaften eines Fehlers korrekt einzuordnen. Nicht jeder Fehler kann durch dieselben Maßnahmen behandelt werden. Während einige Fehler durch eine erneute Ausführung einer Operation behoben werden können, sind andere dauerhaft und erfordern eine Anpassung der Eingabedaten oder eine manuelle Intervention. Die Klassifikation von Fehlern bildet daher die Grundlage für Entscheidungen über Retry-Strategien, Fehlerbehandlung und Wiederherstellungsmechanismen (Newman 2026, Kap. 3 Abschnitt „Should You Always Retry?“).

Darüber hinaus können nicht alle denkbaren Fehler mit vertretbarem Aufwand behandelt werden. Während kurzzeitige Infrastrukturprobleme, Prozessabstürze oder Kommunikationsfehler häufig durch geeignete Architekturmechanismen abgefangen werden können, liegen andere Ereignisse, etwa großflächige Naturkatastrophen oder der vollständige Ausfall eines Rechenzentrums, gegebenenfalls außerhalb des Fehlerannahmemodells einer Anwendung. Der Entwurf fehlertoleranter Systeme erfordert daher eine bewusste Entscheidung darüber, welche Fehlerarten berücksichtigt werden und welche Risiken akzeptiert werden müssen (Kleppmann 2017, Kap. 1 Abschnitt „Reliability“).

Eine grundlegende Einteilung erfolgt in transiente und permanente Fehler. Transiente Fehler treten zeitlich begrenzt auf und verschwinden nach kurzer Zeit wieder. Ursachen liegen häufig in der technischen Infrastruktur eines Systems, beispielsweise in vorübergehenden Netzwerkstörungen, kurzzeitigen Überlastungen von Diensten, nicht erreichbaren Datenbanken oder dem Neustart einzelner Systemkomponenten (Newman 2026, Kap. 3 Abschnitt „Should You Always Retry?“).

Permanente Fehler hingegen bestehen unabhängig von der Anzahl der Wiederholungsversuche fort. Ihre Ursache liegt häufig in der Anwendung selbst oder in den zu verarbeitenden Daten. Beispiele hierfür sind ungültige Eingabedaten oder fehlende Berechtigungen. Newman (2026, Kap. 3 Abschnitt „Should You Always Retry?“) verdeutlicht dies anhand von HTTP-Fehlercodes wie 404 Not Found oder 403 Forbidden.

Die Abgrenzung zwischen beiden Fehlerarten ist insbesondere für die automatisierte Fehlerbehandlung relevant. Während transiente Fehler häufig eine erneute Ausführung rechtfertigen, sollten permanente Fehler möglichst früh erkannt und gesondert behandelt werden.

Eine besondere Rolle spielen zudem Timeout-Fehler. In verteilten Systemen werden Timeouts verwendet, um potenzielle Ausfälle zu erkennen. Dabei kann jedoch nicht eindeutig unterschieden werden, ob tatsächlich ein Fehler vorliegt oder die Antwort lediglich ungewöhnlich lange braucht. Ein Timeout signalisiert daher nur, dass eine erwartete Antwort nicht innerhalb eines definierten Zeitfensters eingetroffen ist und lässt somit keine Aussage über eine Ursache zu (Bass 2021, Kap. 17.2 Abschnitt „Failure in the Cloud“).

2.3.2 Retry-/Backoff-Strategien

Die in Kapitel 2.3.1 eingeführte Unterscheidung zwischen transienten und permanenten Fehlern bildet die Grundlage für den Einsatz von Wiederholungsmechanismen. Da transiente Fehler nur zeitweise die Verarbeitung behindern, kann in solchen Fällen eine erneute Ausführung einer fehlgeschlagenen Operation erfolgreich sein, obwohl der ursprüngliche Versuch fehlgeschlagen ist (Newman 2026, Kap. 3 Abschnitt „Should You Always Retry?“). Retry-Strategien verfolgen damit also das Ziel, die Erfolgswahrscheinlichkeit einer Verarbeitung zu erhöhen, ohne dass ein manueller Eingriff erforderlich wird. Retries sollten ausschließlich für Fehler durchgeführt werden, deren Ursache außerhalb der eigentlichen Geschäftslogik liegt und deren Fortbestand nicht dauerhaft erwartet wird. Permanente Fehler sollten dagegen unmittelbar als endgültig fehlgeschlagen behandelt werden, da eine erneute Ausführung die Erfolgswahrscheinlichkeit nicht erhöht. Ein typisches Szenario, in dem Retries sinnvoll sind, ist das Verschicken von Nachrichten

(Newman 2026, Kap. 3 Abschnitt „Should You Always Retry?“; Microsoft Corporation (Hrsg.) 2026a, Abschnitt „Reliability considerations“).

Ein mögliches Vorgehen bei einem temporären Fehler ist in Abbildung 2.3 dargestellt. Da die Ursache eines transienten Fehlers potenziell auch für eine gewisse Zeit nach dem Auftreten des Fehlers weiterhin besteht, werden Wiederholungen üblicherweise nicht unmittelbar durchgeführt. Stattdessen wird zwischen den einzelnen Versuchen eine Wartezeit eingefügt. Dieses Vorgehen wird als Backoff bezeichnet und soll verhindern, dass wiederholte Fehlversuche zusätzliche Last auf bereits beeinträchtigte Systeme erzeugen. Bekommt eine Anfrage beispielsweise ein Timeout, weil der Server zu viele Anfragen erhält, würden direkte Retries den Server noch weiter mit Anfragen verstopfen. Eine einfache Variante besteht darin, zwischen allen Wiederholungsversuchen dieselbe Wartezeit zu verwenden. In verteilten Systemen wird jedoch häufig ein exponentieller Backoff eingesetzt, bei dem sich das Warteintervall nach jedem fehlgeschlagenen Versuch schrittweise erhöht. Dadurch erhält das betroffene System mehr Zeit, sich von einer temporären Störung zu erholen, da die Dichte der Wiederholungsversuche reduziert wird (Newman 2026, Kap. 3 Abschnitt „Delays Between Retries“).

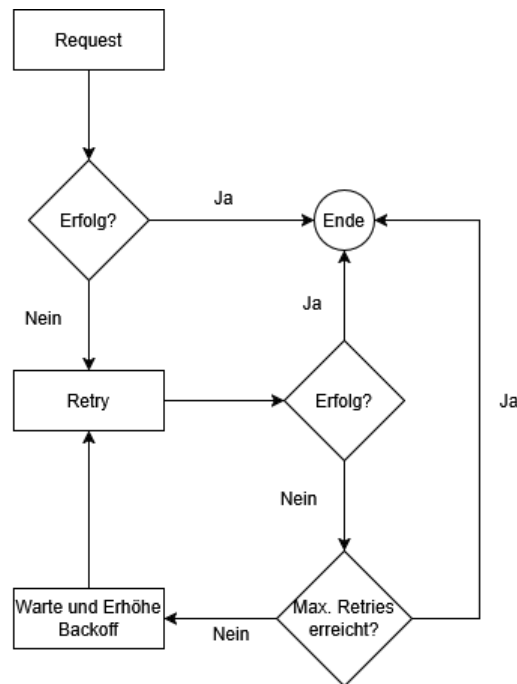


Abbildung 2.3: Verhalten bei transienten Fehlern

Neben der zeitlichen Staffelung von Wiederholungsversuchen muss auch deren Anzahl begrenzt werden. Andernfalls besteht die Gefahr, dass ein fehlerhafter Job unendlich oft ausgeführt wird und dadurch dauerhaft Ressourcen belegt. In der Praxis wird daher eine maximale Anzahl von Wiederholungsversuchen definiert. Wird diese überschritten, gilt die Verarbeitung als endgültig fehlgeschlagen und der Job wird in einen entsprechen-

den Fehlerzustand überführt (Newman 2026, Kap. 3 Abschnitt „How Many Retries Are Appropriate?“).

Retry- und Backoff-Strategien bilden damit einen wesentlichen Baustein zur Behandlung transienter Fehler. Sie ermöglichen die automatische Wiederholung fehlgeschlagener Verarbeitungen, ohne abhängige Systeme durch unmittelbar aufeinanderfolgende Wiederholungsversuche zusätzlich zu belasten.

2.3.3 Recovery

Neben transienten Fehlern, die häufig durch Retry-Mechanismen behandelt werden können, können auch Fehler auftreten, die den Ausführungsprozess einer Verarbeitung vollständig unterbrechen. Besondere Relevanz besitzen Ausfälle ausführender Komponenten während einer laufenden Bearbeitung. In einem solchen Szenario kann kein lokaler Retry mehr initiiert werden, da die ausführende Komponente selbst nicht mehr verfügbar ist. Recovery bezeichnet in diesem Kontext die Fähigkeit eines Systems, eine derart unterbrochene Verarbeitung zu erkennen und geeignete Maßnahmen zur Wiederaufnahme einzuleiten (Bass 2021, Kap. 4.2).

Die Notwendigkeit solcher Mechanismen ergibt sich aus den Eigenschaften verteilter Systeme. Da einzelne Komponenten unabhängig voneinander ausfallen können, während andere Teile des Systems weiterhin funktionieren, entstehen sogenannte Teilausfälle (Kleppmann 2017, Kap. 8 Abschnitt „Faults and Partial Failures“). Dadurch entsteht Unsicherheit über den tatsächlichen Verarbeitungszustand einer Aufgabe. Gegebenenfalls ist nicht eindeutig feststellbar, ob eine Verarbeitung bereits erfolgreich abgeschlossen wurde oder aber die Bestätigung darüber fehlt, ob sie kurz vor ihrem Abschluss steht oder aufgrund eines Fehlers vorzeitig abgebrochen wurde (Kleppmann 2017, Kap. 8).

Zur Wiederherstellung nach einem Ausfall muss zwischen flüchtigem und persistentem Zustand unterschieden werden. Während lokal im Arbeitsspeicher gehaltene Informationen beim Absturz eines Prozesses verloren gehen, kann persistent gespeicherter Zustand auch nach einem Ausfall weiterhin genutzt werden (Kleppmann 2017, Kap. 11 Abschnitt „Rebuilding state after a failure“). Die Verarbeitung auf Basis eines Zwischenzustands wieder aufzunehmen ist unter Umständen jedoch nicht trivial. Eine verbreitete Wiederherstellungsstrategie besteht daher darin, die für eine Verarbeitung relevanten Eingabedaten mindestens bis zum erfolgreichen Abschluss des Jobs zu speichern und eine unterbrochene Verarbeitung bei Bedarf auf Basis dieser Ursprungsdaten erneut auszuführen. Dies ist häufig einfacher und robuster als der Versuch, den exakten Laufzeitzustand eines abgestürzten Prozesses zu rekonstruieren und die Verarbeitung dort fortzusetzen (Bass 2021, Kap. 17.3). Eine erneute Ausführung setzt voraus, dass die zur Verarbeitung benötigten Informationen auch nach einem Ausfall weiterhin verfügbar sind. Darüber hinaus darf ein Ausfall nicht zu inkonsistenten Zuständen führen, die eine spätere Wiederaufnahme verhindern bzw. das Ergebnis beeinflussen. Recovery-Mechanismen werden daher häufig mit Verfahren kombiniert, die eine konsistente Speicherung relevanter Zustandsinformationen gewährleisten (Bass 2021, Kap. 4.2 Abschnitt „Recover from Faults“).

Voraussetzung für eine Wiederaufnahme ist zunächst die Erkennung unterbrochener Verarbeitungen. Wird eine Verarbeitung nach ihrer Übernahme durch eine ausführende

Komponente unterbrochen, kann ein Zustand entstehen, in dem die Aufgabe aus Sicht des Systems weiterhin als aktiv gilt, obwohl tatsächlich keine Bearbeitung mehr stattfindet. Zur Erkennung solcher Situationen werden häufig Heartbeat- oder Lease-/Timeout-Mechanismen eingesetzt (Bass 2021, Kap. 17.2 Abschnitt „Failure in the Cloud“). Hierbei sendet die ausführende Komponente in regelmäßigen Abständen Signale, anhand derer ihre Aktivität überwacht werden kann. Bleiben diese Signale über einen definierten Zeitraum aus, wird ein Ausfall vermutet und die Verarbeitung als potenziell unterbrochen betrachtet (Bass 2021, Kap. 4.2 Abschnitt „Detect Faults“).

Nach der Erkennung eines Ausfalls muss die Verarbeitung erneut angestoßen werden. Hierfür wird die betroffene Aufgabe erneut zur Bearbeitung bereitgestellt, indem sie wieder freigegeben wird. Da eine Verarbeitung häufig erst nach einer expliziten Erfolgsquittierung als abgeschlossen gilt, führt das Ausbleiben einer solchen Quittung infolge eines Ausfalls dazu, dass die Aufgabe erneut verarbeitet wird (Microsoft Corporation (Hrsg.) 2026a).

Für Recovery sind also zwei Schritte essenziell. Zunächst muss erkannt werden, dass eine Verarbeitung unterbrochen wurde. Anschließend muss die Verarbeitung neu angestoßen werden. Dafür existieren unterschiedliche Ansätze. Einerseits kann ein System Zwischenzustände persistent speichern und die Verarbeitung nach einem Ausfall auf Basis des zuletzt bekannten Zustands fortsetzen. Andererseits kann die Verarbeitung vollständig neu gestartet werden, sofern die dafür benötigten Eingabedaten weiterhin verfügbar sind. Welcher Ansatz gewählt wird, hängt unter anderem von der Komplexität der Verarbeitung sowie vom Aufwand einer vollständigen Neuausführung ab.

Die in diesem Kapitel beschriebene Wiederaufnahme führt zwangsläufig dazu, dass einzelne Verarbeitungsschritte mehrfach ausgeführt werden können. Da dabei häufig nicht eindeutig festgestellt werden kann, ob eine vorherige Ausführung bereits teilweise oder vollständig erfolgreich war, müssen Systeme auf mögliche Mehrfachausführungen vorbereitet sein. Hieraus ergibt sich die Notwendigkeit idempotenter Verarbeitung, die im folgenden Abschnitt näher betrachtet wird.

2.3.4 Idempotenz

Die in den vorherigen Abschnitten beschriebenen Retry- und Wiederherstellungsmechanismen erhöhen die Zuverlässigkeit eines Systems, führen jedoch gleichzeitig dazu, dass einzelne Verarbeitungsschritte mehrfach ausgeführt werden können. Um trotz solcher Mehrfachausführungen konsistente Datenbestände sicherzustellen, wird das Konzept der Idempotenz eingesetzt. Eine Operation wird als idempotent bezeichnet, wenn ihre mehrmalige Ausführung denselben fachlichen Zustand erzeugt wie eine einmalige Ausführung (Newman 2026, Kap. 3 Abschnitt „Idempotency“). Wiederholte Aufrufe verändern das Ergebnis somit nicht, wodurch Duplikate oder erneut ausgeführte Verarbeitungsschritte keine inkonsistenten Zustände verursachen. Wie in Kapitel 2.1.2 dargelegt, stellt Idempotenz damit eine wesentliche Voraussetzung dar, um auf Basis einer At-Least-Once-Semantik eine Exactly-Once-Semantik zu erreichen.

Idempotenz kann dabei auf unterschiedlichen Ebenen eines Systems umgesetzt werden. In Bezug auf Job-Processing muss bereits die Aufnahme eines Jobs in den Job-Store

idempotent gestaltet werden, um zu verhindern, dass derselbe Job mehrfach im System erfasst wird. Ein verbreiteter Ansatz zur idempotenten Annahme von Anfragen ist die Implementierung eines idempotenten Receivers (Hohpe und Woolf 2005, Kap. 10 Abschnitt „Idempotent Receiver“) unter Verwendung eindeutiger Identifikatoren, wie beispielsweise eines Universally Unique Identifier (UUID). Bei einer möglichen Variante wird jeder Anfrage vom Sender eine eindeutige Kennung zugefügt, die vom Empfänger zusammen mit der Anfrage persistent gespeichert wird. Geht dieselbe Anfrage erneut ein, kann das System anhand dieser UUID erkennen, dass die Anfrage bereits verarbeitet wurde und eine redundante Speicherung verhindern (Newman 2026, Kap. 3 Abschnitt „Client Request IDs“; Richardson 2018, Kap. 3.3.6).

Darüber hinaus muss auch die eigentliche Verarbeitung eines Jobs so gestaltet werden, dass wiederholte Ausführungen und Wiederaufnahmen keine inkonsistenten Ergebnisse erzeugen (Newman 2026, Kap. 3 Abschnitt „Idempotency“). Statusänderungen und fachliche Datenänderungen müssen folglich über klar definierte Transaktionsgrenzen abgesichert werden, um eine konsistente Weiterbearbeitung nach einem Crash zu gewährleisten (Kleppmann 2017, Kap. 7).

Idempotenz bildet somit die Grundlage dafür, dass Retry- und Recovery-Mechanismen eingesetzt werden können, ohne die Konsistenz der verarbeiteten Daten zu gefährden. Sie stellt einen zentralen Baustein fehlertoleranter Background-Job-Processing-Systeme dar und ermöglicht die korrekte Verarbeitung trotz unvermeidbarer Mehrfachausführungen und Duplikate.

2.4 Stand der Forschung

Die Verarbeitung von Hintergrundaufgaben ist ein etabliertes Problemfeld verteilter Systeme. Entsprechend existieren zahlreiche Frameworks und Architekturmuster, die eine asynchrone Hintergrundaufführung von Aufgaben ermöglichen. Die Ansätze unterscheiden sich insbesondere hinsichtlich der verwendeten Infrastruktur, der Persistenzmechanismen sowie der Umsetzung von Fehlertoleranz und Wiederherstellung. Im Folgenden werden die für diese Arbeit relevanten Lösungsansätze vorgestellt und hinsichtlich ihrer Eigenschaften eingeordnet.

2.4.1 Brokerbasierte Ansätze

Eine weit verbreitete Klasse von Background Job-Processing-Systemen basiert auf dedizierten Message Brokern wie RabbitMQ (Broadcom Inc. (Hrsg.) 2026a) oder Apache Kafka (Apache Software Foundation (Hrsg.) 2026). Hierbei werden Aufgaben in Form von Nachrichten an einen Broker übergeben, der die Entkopplung zwischen Produzenten und Konsumenten übernimmt. Worker beziehen die Nachrichten über den Broker und führen die zugehörige Verarbeitung aus. Der Broker läuft als eigenständige Middleware-Komponente entweder nativ oder virtuell als Server. Der Vorteil dieses Ansatzes liegt in der hohen Skalierbarkeit und der klaren Trennung zwischen Anwendungslogik und Nachrichteninfrastruktur. Darüber hinaus stellen moderne Broker Mechanismen zur Fehlertoleranz, Lastverteilung und Ausfallsicherheit bereit. Gleichzeitig entsteht durch broker-

basierte Architekturen zusätzliche betriebliche Komplexität, da neben der eigentlichen Anwendung weitere Infrastrukturkomponenten aufgesetzt, betrieben und überwacht werden müssen. Des Weiteren ergeben sich Herausforderungen bei der Konsistenz zwischen Anwendungsdatenbank und Message Broker. Problematisch ist beispielsweise die atomare Kopplung von Datenänderungen in der Datenbank und das Verschicken einer Nachricht. Da beide Systeme unabhängig voneinander arbeiten, müssen zusätzliche Mechanismen wie das Transactional Outbox Pattern eingesetzt werden, um Nachrichtenverlust oder Inkonsistenzen zu vermeiden (Richardson 2018, Kap. 3.3.7).

2.4.2 Datenbankbasierte Ansätze

Neben brokerbasierten Lösungen existieren Frameworks, die eine relationale Datenbank als zentralen Job-Store verwenden. Zu bekannten Vertretern dieses Ansatzes zählen unter anderem JobRunr (Rosoco BV (Hrsg.) 2026), Hangfire (Hangfire OÜ (Hrsg.) 2026) oder Spring Batch (Broadcom Inc. (Hrsg.) 2026b). Anstatt Aufgaben an einen separaten Broker zu übertragen, werden diese als persistente Datensätze in einer Datenbank gespeichert und von Workern direkt ausgelesen.

Die Nutzung einer relationalen Datenbank ermöglicht den Gebrauch der ACID-Eigenschaften, um das Speichern neuer Jobs sowie das Verwalten von Status- und fachlichen Datenänderungen konsistent innerhalb einer gemeinsamen Transaktion zu verarbeiten. In der Praxis verfolgen existierende Frameworks jedoch unterschiedliche Schwerpunkte und adressieren die Problemstellung jeweils aus einer anderen Perspektive. Hangfire garantiert nur eine At-Least-Once Ausführung und überlässt die Umsetzung von Idempotenz der Fachlogik. Während der Quartz Scheduler (Terracotta Inc. (Hrsg.) 2026) primär auf zeitgesteuertes Scheduling und clusteringbasierte Koordination fokussiert, stellt Spring Batch eine spezialisierte Infrastruktur für die blockweise Massenverarbeitung bereit, die auf periodische oder zeitgesteuerte Durchläufe ausgelegt ist. Modernere Java-Bibliotheken wie JobRunr und Hangfire integrieren zwar direkt ein automatisiertes Retry-Handling, kapseln die zugrundeliegenden Mechanismen jedoch als Blackbox, wodurch eine feingranulare Kontrolle und die präzise Analyse des Systemverhaltens bei Infrastrukturausfällen erschwert werden. Robuste und langlebige Workflow-Ausführungen wie Temporal (Temporal Technologies Inc. 2026) lösen die Frage nach Fehlertoleranz wiederum über eine dedizierte Engine, erzwingen damit jedoch einen eigenständigen Serverbetrieb und eine damit einhergehende infrastrukturelle Komplexität. Die konkrete Umsetzung der Nebenläufigkeitskontrolle, Wiederherstellung und Fehlerbehandlung ist stark von den jeweiligen Frameworks abhängig. Die zugrundeliegenden Prinzipien bleiben somit hinter der Schnittstelle der Frameworks verborgen und lassen sich nicht ohne Weiteres abstrahieren oder auf eigene Systementwürfe übertragen.

2.4.3 Einordnung und Forschungslücke

Die vorgestellten Ansätze zeigen, dass sowohl brokerbasierte als auch datenbankbasierte Architekturen zur Umsetzung von Background-Job-Processing-Systemen geeignet sind. Während bestehende Frameworks bereits umfangreiche Funktionalitäten für Scheduling,

Retry-Mechanismen und Fehlertoleranz bereitstellen, liegt ihr Fokus primär auf der praktischen Nutzung, wodurch die zugrundeliegenden transaktionalen Mechanismen und deren Verhalten in kritischen Ausfallszenarien für den Entwickler oft intransparent bleiben. Insbesondere die Frage, unter welchen Voraussetzungen eine relationale Datenbank gleichzeitig als persistenter Job-Store und Koordinationsmechanismus eingesetzt werden kann, wird in der Literatur nur begrenzt behandelt. Konzepte wie Fehlertoleranz, Recovery, Idempotenz und Nebenläufigkeitskontrolle werden zwar häufig einzeln beschrieben, ihre Integration zu einer konsistenten Gesamtarchitektur bleibt jedoch oftmals implizit.

Die vorliegende Arbeit grenzt sich von bestehenden Frameworks ab, indem sie ein rein datenbankgestütztes Pull-Modell entwirft, die Architekturentscheidungen begründet darlegt und das System bezüglich der angestrebten Fehlertoleranz evaluiert. Durch den Einsatz von SQL-Polling in Kombination mit der PostgreSQL-Funktionalität `FOR UPDATE SKIP LOCKED` und einem Statusmodell wird untersucht, wie die Koordination und Ausfallwiederherstellung direkt auf der Datenbankebene orchestriert werden kann, ohne einen separaten Message Broker einzusetzen.

3 Methodik

3.1 Literaturrecherche

3.2 Ergebnisgenerierung

Problemstellung ↓ Fehlermodell ↓ Anforderungen ↓ Zusicherungen ↓ Architektur ↓ Implementierung ↓ Evaluation

4 Anforderungen

Dieses Kapitel legt die Anforderungen an das im Rahmen der Arbeit entwickelte Background Job-Processing-System fest und schafft damit die Grundlage für den späteren Architekturentwurf sowie dessen Evaluation. Zunächst wird der Systemkontext beschrieben, um die beteiligten Komponenten, Schnittstellen und Verantwortlichkeiten einzuordnen. Anschließend wird das betrachtete Fehlermodell abgegrenzt und damit festgelegt, welche Störungen und Randbedingungen für den Entwurf relevant sind. Darauf aufbauend werden präzise Systemzusicherungen formuliert, die das gewünschte Verhalten unabhängig von konkreten Implementierungsdetails beschreiben und als verbindlicher Maßstab für die Umsetzung und die Evaluation dienen.

4.1 Systemkontext

Im Rahmen dieser Arbeit wird ein datenbankgestütztes Background Job-Processing-System entwickelt, dessen grundlegende Systemstruktur in Abbildung 4.1 dargestellt ist. Über eine REST-Schnittstelle können Aufträge erzeugt werden, die zunächst validiert und anschließend in einem relationalen Job-Store persistent gespeichert werden. Unabhängig vom ursprünglichen Aufruf übernehmen mehrere Worker-Prozesse die asynchrone Verarbeitung der gespeicherten Jobs. Zur Evaluation der entwickelten Architektur

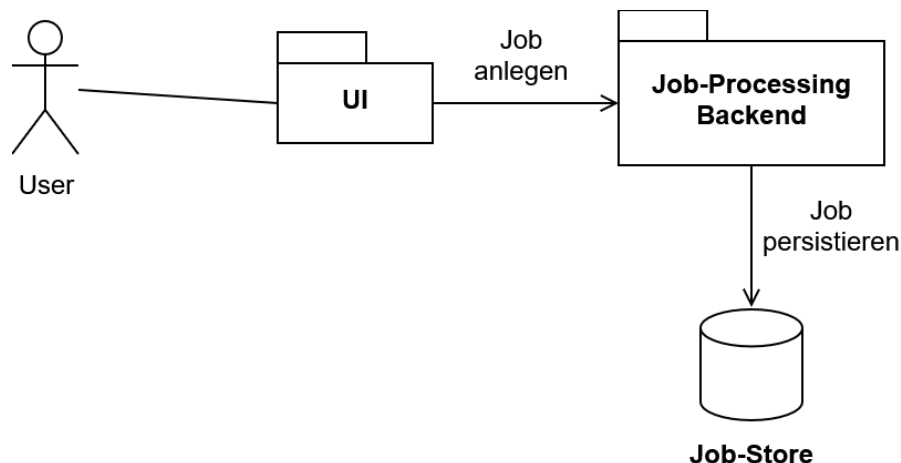


Abbildung 4.1: Queueing

wird exemplarisch ein Anwendungsszenario aus dem Umfeld der Verarbeitung technischer Wartungsdokumente betrachtet. Die konkrete fachliche Verarbeitung dient dabei ausschließlich als Demonstrations- und Evaluationsszenario, während der Schwerpunkt dieser Arbeit auf dem Entwurf der zugrunde liegenden Architektur für eine zuverlässige Hintergrundverarbeitung liegt. Ziel ist somit nicht die Entwicklung einer domänenspezifischen Fachanwendung, sondern einer allgemein einsetzbaren Architektur, deren Konzepte

unabhängig vom gewählten Anwendungskontext auf andere Background Job-Processing-Systeme übertragbar sind.

Das betrachtete System vereint die wesentlichen Herausforderungen datenbankgestützter Background Job-Processing-Systeme. Hierzu zählen insbesondere die zuverlässige Persistierung eingehender Aufträge, die koordinierte Verarbeitung durch mehrere parallel arbeitende Worker, die Behandlung konkurrierender Zugriffe sowie die Wiederaufnahme unterbrochener Verarbeitungen nach Teilausfällen. Die genau betrachtete Fehlermenge wird im folgenden Kapitel dargelegt.

4.2 Abgrenzung des betrachteten Fehlermodells

Verteilte Softwaresysteme können durch eine Vielzahl unterschiedlicher Fehler beeinträchtigt werden. Hierzu zählen unter anderem Hardwaredefekte, Softwarefehler, Netzwerkausfälle, Bedienfehler oder Cyberangriffe. Wie bereits in Kapitel 2.3 erläutert wurde, kann ein System jedoch nicht sämtliche denkbaren Fehler verhindern oder behandeln. Der Entwurf fehlertoleranter Systeme erfordert daher eine bewusste Abwägung, welche Fehlerarten berücksichtigt und welche Risiken akzeptiert werden. Diese Entscheidung orientiert sich zum einen am Anwendungskontext und den Anforderungen an das System und zum anderen an wirtschaftlichen Randbedingungen wie Entwicklungs- und Betriebsaufwand (Kleppmann 2017, Kap. 1 Abschnitt „Reliability“).

Die im Folgenden dargelegte Teilmenge betrachteter Fehler ist eine Auswahl der von Kleppmann (2017), Newman (2026) und Bass (2021) beschriebenen typisch vorkommenden Fehlern in verteilten Systemen. Ziel dieser Arbeit ist der Entwurf einer fehlertoleranten Architektur für ein datenbankgestütztes Background Job-Processing-System. Betrachtet werden daher ausschließlich Fehlerklassen, die während des regulären Betriebs auftreten können und deren Auswirkungen durch Mechanismen der Anwendungsarchitektur beherrscht werden können. Fehler, deren Behandlung zusätzliche Infrastruktur oder organisatorische Maßnahmen wie Backups, Replikation oder physische Redundanz erfordern, liegen außerhalb des Betrachtungsumfangs dieser Arbeit.

Die Abgrenzung der betrachteten Fehler ist in table 4.1 zusammengefasst. Innerhalb des Systementwurfs werden im Wesentlichen vier Kategorien von Fehlern behandelt. An erster Stelle stehen Ausfälle einzelner Systemkomponenten sowie transiente Infrastruktur- und Kommunikationsstörungen. Hierzu zählen insbesondere das unerwartete Beenden eines Worker-Prozesses, Container-Neustarts, JVM-Abstürze, durch das Betriebssystem terminierte Prozesse, temporäre Netzwerkunterbrechungen oder kurzfristig nicht erreichbare Datenbanken. Dadurch kann es im System zu zwei Problemen kommen. Zum einen besteht die Gefahr, dass eine laufende Verarbeitung unterbrochen wird oder vorübergehend nicht fortgesetzt werden kann. Aufgrund der Charakteristik transienter Fehler wird dabei grundsätzlich davon ausgegangen, dass die betroffene Komponente oder Ressource nach kurzer Zeit wieder verfügbar ist. Zum anderen entsteht die Gefahr, dass eine Nachricht bzw. ein Job verloren geht. Durch diese Fehler gerät das Gesamtsystem in einen unklaren Zustand darüber, wie weit die Verarbeitung bereits fortgeschritten war bzw. ob der Job persistiert wurde. Da diese Störungen in verteilten Umgebungen regelmäßig

auftreten können (Newman 2026, Kap. 1 Abschnitt „Failure Is Inevitable“), muss die Systemarchitektur in der Lage sein, diese autonom zu überbrücken. Als direkte Folge

Fehlerklasse	Relevant	Begründung
Komponenten und Infrastrukturausfälle	✓	Unterbrechen die Verarbeitung und gefährden die Zuverlässigkeit des Systems
Doppelte Requests	✓	Können zu mehrfach persistierten Jobs führen
Doppelte Ausführung	✓	Kann inkonsistente Seiteneffekte verursachen
Konkurrierender Zugriff	✓	Gefährdet die Konsistenz gemeinsamer Daten
Physischer Datenbankverlust	✗	Erfordert Backups & Redundanz
Totalausfall Infrastruktur	✗	Erfordern Disaster-Recovery Maßnahmen
Cyberangriffe	✗	Qualitätsmerkmal Sicherheit
Byzantinische Fehler	✗	Annahme: Vertrauensvolle Komponenten

Tabelle 4.1:

solcher Infrastrukturprobleme und der darauf reagierenden Wiederholungsmechanismen der Clients entstehen doppelte Requests. Ein Client sendet eine Anfrage erneut, weil eine Bestätigung aufgrund eines Timeouts ausblieb. Die Architektur muss in diesem Szenario durch Duplikaterkennung verhindern, dass identische Jobs mehrfach im Job-Store persistiert werden.

Eng damit verknüpft ist die doppelte Ausführung von Jobs. Bei der Wiederaufnahme eines zuvor abgebrochenen Jobs kann nach einem Fehler oft nicht eindeutig festgestellt werden, ob die vorherige Verarbeitung bereits Seiteneffekte erzielt hat. Die Architektur muss daher zwingend sicherstellen, dass wiederholte Ausführungen keine inkonsistenten Ergebnisse verursachen.

Des Weiteren werden konkurrierende Zugriffe betrachtet. Da die Parallelverarbeitung mittels mehrerer, autonom agierender Worker ein wesentliches Merkmal moderner Background Job-Processing-Systeme darstellt, entstehen unweigerlich gleichzeitige Zugriffe auf geteilte Datenstrukturen wie den zentralen Job-Store. Dies ist im klassischen Sinne zwar nicht direkt ein Fehler, jedoch können parallele Zugriffe ohne entsprechende Synchronisationsmechanismen potenziell zu Race Conditions und inkonsistenten Zuständen in der Persistenzschicht führen, weshalb deren Verhinderung wesentlicher Bestandteil des Entwurfs ist.

Dem gegenüber stehen Fehler, die bewusst außerhalb des Betrachtungsumfangs dieser Arbeit liegen. Ein physischer Datenbankverlust sowie ein Totalausfall der Infrastruktur

können nicht durch die Softwarearchitektur des Job-Processing-Systems gelöst werden. Ein Verlust der Persistenzschicht oder ein Ausfall des gesamten Rechenzentrums infolge von großflächigen Stromausfällen oder Bränden am Serverstandort erfordert stattdessen infrastrukturelle Maßnahmen wie geografische Redundanz, kontinuierliche Backups und umfassende Disaster-Recovery-Konzepte.

Ebenso werden Cyberangriffe im Rahmen dieser Arbeit ausgeschlossen. Obwohl Angriffe wie SQL-Injections oder Denial-of-Service-Szenarien direkte Auswirkungen auf die Zuverlässigkeit eines Systems haben können, betreffen sie primär das Qualitätsmerkmal der Sicherheit. Ihre Abwehr erfordert spezialisierte Mechanismen wie strikte Eingabevalidierung, Ratenbegrenzungen, Authentifizierung und Angriffserkennungssysteme. Der Fokus dieser Arbeit liegt stattdessen auf der Gewährleistung von Fehlertoleranz bei regulärem Systembetrieb.

Schließlich werden auch byzantinische Fehler ausgeschlossen. Dies sind Fehler, bei denen kompromittierte oder defekte Komponenten willkürlich falsche oder manipulierte Informationen erzeugen und verbreiten (Kleppmann 2017, Kap. 8 Abschnitt „Byzantine Faults“). Die vorliegende Arbeit geht dahingehend von der Annahme ehrlicher Komponenten aus. Dies bedeutet, dass Komponenten zwar abstürzen oder vorübergehend nicht erreichbar sein können, im aktiven Zustand jedoch nie absichtlich falsche Informationen verbreiten.

Aus der gewählten Abgrenzung ergibt sich, dass der Schwerpunkt dieser Arbeit auf der Behandlung von Teilausfällen, konkurrierender Verarbeitung und transienten Infrastrukturfehlern liegt. Diese Fehlerszenarien besitzen unmittelbaren Einfluss auf die Zuverlässigkeit datenbankgestützter Background Job-Processing-Systeme und bilden daher zusammen mit der Systembeschreibung aus Kapitel 4.1 die Grundlage für die im Folgenden formulierten Systemzusicherungen.

4.3 Zusicherungen

Aus dem in Kapitel 4.1 beschriebenen Systemkontext und dem in Kapitel 4.2 definierten Fehlermodell ergeben sich Eigenschaften, die das entwickelte Background Job-Processing-System zu jedem Zeitpunkt gewährleisten muss, um die fehlerfreie Funktion des Systems bezogen auf die Zielsetzung der Arbeit sicherzustellen. Diese Eigenschaften werden im Folgenden als Systemzusicherungen bezeichnet.

Im Gegensatz zu konkreten Architekturmechanismen beschreiben Systemzusicherungen ausschließlich das von der Architektur einzuhaltende Verhalten. Sie formulieren Invarianten des Systems, die unabhängig von der jeweiligen Implementierung oder dem aktuell betrachteten Fehlerszenario jederzeit gelten müssen. Die in Kapitel 5.1 entwickelten Architekturmechanismen dienen dazu, die Einhaltung dieser Zusicherungen sicherzustellen.

4.3.1 Zuverlässige Annahme von Jobs

Da Jobs zeitlich entkoppelt verarbeitet werden, bildet ihre persistente Speicherung die Grundlage sämtlicher weiterer Verarbeitungsschritte. Bereits bei der Annahme muss daher sichergestellt werden, dass dem Client ausschließlich solche Jobs als erfolgreich bestä-

tigt werden, die dauerhaft gespeichert wurden. Gleichzeitig können durch Kommunikationsfehler oder Wiederholungsmechanismen identische Anfragen mehrfach beim System eintreffen. Daraus ergeben sich die folgenden Zusicherungen:

- Z1: Persistenz bestätigter Jobs** Jeder Job, dessen Annahme gegenüber dem Client bestätigt wurde, ist dauerhaft im System vorhanden und geht nicht verloren.
- Z2: Korrekte Annahmestätigung** Eine erfolgreiche Annahmestätigung erfolgt ausschließlich dann, wenn der Job erfolgreich persistiert wurde.
- Z3: Idempotente Annahme** Zu jedem logischen Auftrag existiert höchstens ein persistierter Job.

4.3.2 Konsistenz der Verarbeitung

Mehrere parallel arbeitende Worker sowie mögliche Wiederholungen infolge von Teilausfällen haben Auswirkungen auf die Konsistenz der Verarbeitung. Unabhängig davon, wie häufig ein Job erneut ausgeführt oder von unterschiedlichen Komponenten verarbeitet wird, muss der interne Zustand des Systems jederzeit widerspruchsfrei bleiben. Außerdem muss das System die Zustände der Jobs kennen, damit ausschließlich ausstehende und nicht derzeit ausgeführte Jobs verarbeitet werden. Um fachliche Fehler in den Daten aufgrund von parallelen Arbeitseinheiten und Teilausfällen zu vermeiden, ergeben sich folgende Zusicherungen:

- Z4: Eindeutiger Verarbeitungszustand** Jeder Job befindet sich zu jedem Zeitpunkt in genau einem gültigen Verarbeitungszustand.
- Z5: Gültige Zustandsübergänge** Statusübergänge erfolgen ausschließlich entsprechend der definierten Übergänge.
- Z6: Exklusive Verarbeitung** Ein Job wird zu keinem Zeitpunkt gleichzeitig von mehreren Workern verarbeitet.
- Z7: Atomare Wirkung** Nach außen sind ausschließlich vollständig abgeschlossene Ergebnisse sichtbar.
- Z8: Erfolgseinmaligkeit** Zu jedem Job wird höchstens ein fachliches Endergebnis dauerhaft persistiert.

4.3.3 Fortsetzung der Verarbeitung

Das betrachtete Fehlermodell umfasst Ausfälle einzelner Worker sowie transiente Infrastrukturstörungen. Solche Fehler dürfen zwar die Bearbeitung eines Jobs unterbrechen, jedoch nicht dauerhaft verhindern. Das System muss daher sicherstellen, dass begonnene Verarbeitungen abgeschlossen werden können. Hieraus ergeben sich folgende Zusicherungen:

Z9: Wiederherstellbarkeit Ein begonnener Job geht aufgrund eines Teilausfalls nicht dauerhaft verloren.

Z10: Terminierung Jeder erfolgreich angenommene Job erreicht nach endlicher Zeit einen Endzustand (SUCCEEDED oder FAILED).

5 Umsetzung

5.1 Architektur und Design

5.1.1 Architekturentwurf

Eine verbreitete Variante, das Queue-Based Load Leveling Pattern für Nachrichten, oder wie im Fall dieser Arbeit für Jobs, umzusetzen, ist die Implementierung mithilfe dedizierter Message Broker (Kleppmann 2017, Kap. 11 Abschnitt „Message brokers“). Damit keine Jobs verloren gehen und keine doppelten oder inkonsistenten Ergebnisse entstehen, strebt das System eine Exactly-Once Semantik an. Für das im Rahmen dieser Arbeit entwickelte System wird daher bewusst eine relationale PostgreSQL-Datenbank als zentraler Job-Store gewählt. Durch die Nutzung PostgreSQL-spezifischer Mechanismen wie *SKIP LOCKED* in Kombination mit den vollständigen ACID-Transaktionsgarantien relationaler Datenbanken lässt sich ein Großteil der in Kapitel 4 erhobenen Anforderungen mittels der inhärenten Datenbankfunktionen realisieren. Die Datenbank übernimmt dabei nicht ausschließlich die Rolle eines persistenten Datenspeichers, sondern fungiert gleichzeitig als Warteschlange sowie als Synchronisationsmechanismus zwischen konkurrierenden Workern. Funktional orientiert sich die Architektur somit an den Prinzipien eines klassischen Message Brokers, nutzt jedoch die Konsistenzgarantien einer relationalen Datenbank als Grundlage für die Umsetzung der definierten Zusicherungen.

Die Wahl eines datenbankbasierten Job-Stores stellt einen bewussten Architektur-Trade-off dar. Im Vergleich zu In-Memory-basierten Messaging-Systemen entstehen durch den permanenten Zugriff auf einen externen persistenten Datenspeicher zusätzliche Latenzen und ein geringerer maximaler Nachrichtendurchsatz. Demgegenüber ermöglicht die relationale Datenbank jedoch eine dauerhafte Speicherung sämtlicher Jobs sowie atomare Zustandsübergänge und eine konsistente Wiederherstellung des Systemzustands nach Fehlern. Zusätzlich bietet eine relationale Datenbank mehr Sicherheiten vor verloren gegangenen Jobs, als typische In-Memory Message Broker, die dafür aufwändige Mechanismen benötigen. Die Haltung des Jobstatus in einer Datenbank hat außerdem für sehr rechenintensive Operationen den Vorteil, dass eine angefangene Operation nach einem Teilausfall sehr einfach fortgesetzt werden kann, anstatt den Job vollständig neu starten zu müssen, indem vor der Wiederaufnahme der letzte bekannte Status in der Datenbank nachgeschaut wird (Kleppmann 2017, Kap. 11, Abschnitt „Rebuilding state after a failure“). Gerade für kleine bis mittelgroße Systeme stellt dies eine geeignete und praktikable Architektur dar, wenn ein Jobverlust nicht in Kauf genommen werden kann, da auf eine aufwändige Clusterstruktur zur Absicherung von Jobverlusten verzichtet werden kann (Newman 2026, Kap. 8 Abschnitt „HowBrokers Work (In A Nutshell)“).

Die resultierende Gesamtarchitektur ist in Abbildung 5.1 dargestellt. Der Client bildet den Einstiegspunkt des Systems und erzeugt neue Jobs, die über eine REST-Schnittstelle an das Backend übertragen werden. Im Rahmen des entwickelten Prototyps erfolgt dies lediglich direkt über einen Service, der die Kommunikation mit dem Backend-Application Programming Interface (API) kapselt. In der Praxis kann diese Komponente jedoch eben-

so durch eine grafische Benutzeroberfläche oder ein beliebiges externes Anwendungssystem angesprochen werden. Die REST-API dient als Schnittstelle des Backends zur Erstel-

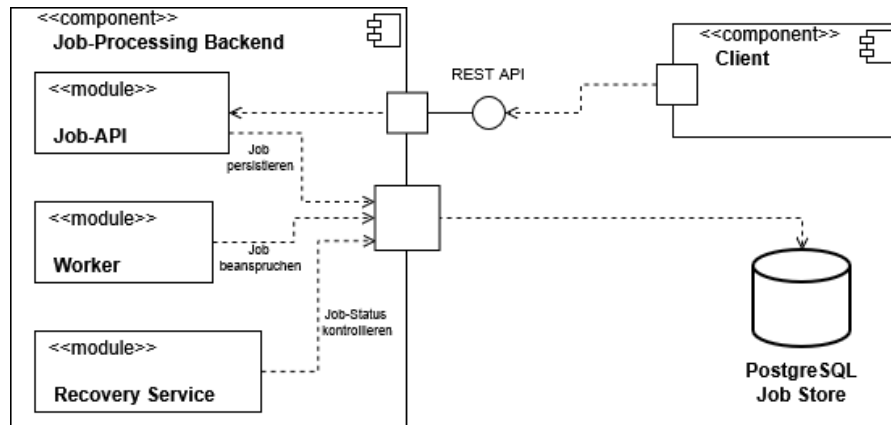


Abbildung 5.1: Abstrakte Architektur des entwickelten Systems

lung neuer Jobs. Sie nimmt eingehende Anfragen entgegen, validiert diese und delegiert sie an die Servicelogik, welche für die Persistierung neuer Jobs sowie die Verwaltung ihrer Zustände verantwortlich ist. Alle Jobs werden anschließend im zentralen Job-Store gespeichert, der neben den eigentlichen Jobdaten auch den aktuellen Bearbeitungszustand sowie gegebenenfalls erzeugte Ergebnisse enthält.

Die eigentliche Verarbeitung erfolgt durch mehrere parallel arbeitende Worker. Jeder Worker beansprucht einen Job exklusiv aus dem Job-Store, verarbeitet dessen Nutzdaten und speichert das erzeugte Ergebnis anschließend wieder in der Datenbank. Da sämtliche Worker auf denselben Job-Store zugreifen, kann die Verarbeitung horizontal skaliert werden, ohne dass eine direkte Kommunikation zwischen den einzelnen Workern erforderlich ist.

Ergänzt wird die Architektur durch einen Recovery-Service, der den Job-Store periodisch auf verwaiste Jobs überprüft. Kann ein Worker einen bereits beanspruchten Job aufgrund eines Fehlers oder Absturzes nicht erfolgreich abschließen, erkennt der Recovery-Service anhand des abgelaufenen Lease, dass die Bearbeitung nicht abgeschlossen wurde. Der Status des betroffenen Jobs wird, sofern der Recovery-Service eine erneute Bearbeitung für sinnvoll erachtet, zurückgesetzt und steht damit einem beliebigen Worker erneut zur Bearbeitung zur Verfügung. Auf diese Weise wird verhindert, dass Jobs dauerhaft im Zustand **RUNNING** verbleiben oder infolge eines Worker-Ausfalls verloren gehen.

5.1.2 Zustandsmodell

Zur Umsetzung der in Kapitel 4 definierten Zusicherungen Z4 und Z5 wird jeder Job durch ein explizites Zustandsmodell beschrieben. Der aktuelle Zustand eines Jobs wird dabei als Attribut des jeweiligen Datensatzes in der Datenbank gespeichert (vgl. Kapitel 5.1.3) und beschreibt den Fortschritt der Verarbeitung. Das Zustandsmodell bildet die Grundlage für

sämtliche Entscheidungen bezüglich der Bearbeitung, Wiederherstellung und Beendigung eines Jobs.

Ein wesentlicher Vorteil dieses Ansatzes besteht darin, dass Datenbankzeilen nicht über den gesamten Verarbeitungszeitraum exklusiv gesperrt werden müssen. Sperren sind lediglich während kurzer atomarer Zustandsübergänge erforderlich, während die eigentliche Verarbeitung außerhalb einer Datenbanktransaktion erfolgt. Dadurch werden konkurrierende Zugriffe auf den Job-Store reduziert und Datenbankressourcen geschont, was sich positiv auf die Skalierbarkeit des Systems auswirkt.

Die in diesem System definierten Zustände sowie die zulässigen Zustandsübergänge sind in Abbildung 5.2 dargestellt. Nachdem ein neuer Job über die REST-Schnittstelle im

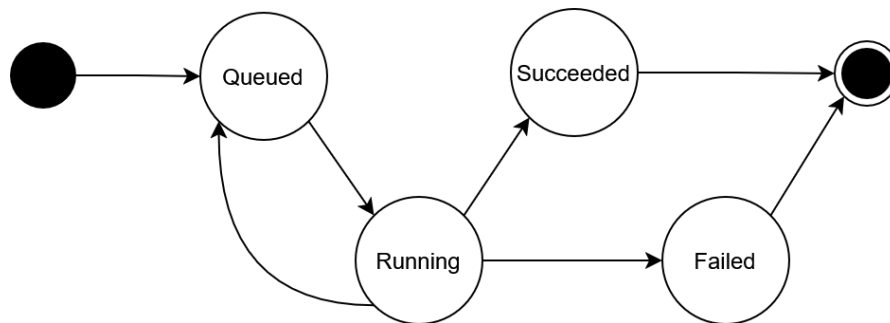


Abbildung 5.2: Zustandsmodell eines Jobs

Backend eingegangen ist, wird dieser persistent gespeichert und zunächst in den Zustand *Queued* versetzt. In diesem Zustand wartet der Job auf seine Bearbeitung und kann von einem Worker beansprucht werden. Beansprucht ein Worker den Job erfolgreich, erfolgt der atomare Zustandsübergang nach *Running*. Dadurch wird sichergestellt, dass der Job während seiner Verarbeitung keinem weiteren Worker zur Verfügung steht.

Zu der Verarbeitung existieren drei mögliche Folgezustände. Kann der Worker den Job erfolgreich bearbeiten und das erzeugte Ergebnis dauerhaft in der Datenbank speichern, wird der Job in den Endzustand *Succeeded* überführt. Tritt während der Verarbeitung hingegen ein Fehler auf oder beendet der Worker seine Arbeit nicht innerhalb der vorgesehenen Lease-Dauer, übernimmt der Recovery-Service die weitere Behandlung des Jobs. Ist gemäß der definierten Recovery-Strategie ein erneuter Ausführungsversuch zulässig, wird der Job zurück in den Zustand *Queued* versetzt und kann anschließend erneut von einem beliebigen Worker übernommen werden. Wurde dagegen die maximale Anzahl zulässiger Wiederholungsversuche erreicht oder liegt ein nicht behebbarer Fehler vor, wird der Job endgültig in den Endzustand *Failed* überführt. Jobs in diesem Zustand werden vom System nicht weiter automatisch verarbeitet und erfordern das manuelle Eingreifen durch den Benutzer oder Administrator.

5.1.3 Entwurf der Datenbank

Die relationale Datenbank bildet den zentralen Job-Store des entwickelten Systems. Sie übernimmt sowohl die Persistierung eingehender Jobs als auch die Verwaltung ihres ak-

tuellen Bearbeitungszustands und gegebenenfalls erzeugter Verarbeitungsergebnisse. Das resultierende Datenbankschema ist in Abbildung 5.3 dargestellt. Hauptbestandteil des

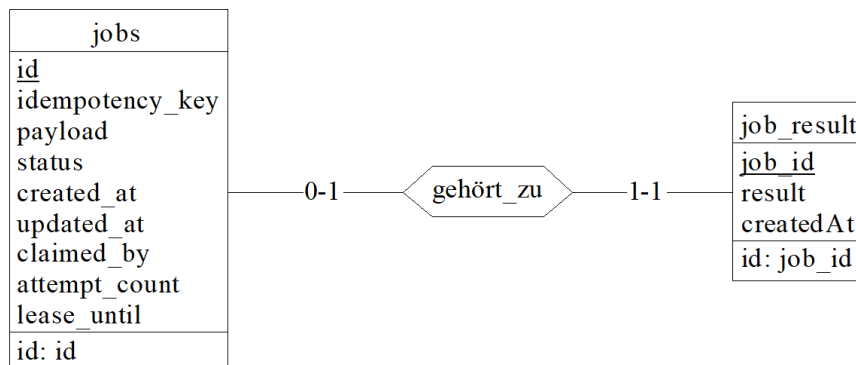


Abbildung 5.3: Datenbankschema des entwickelten Job-Processing-Systems

Schemas ist die Tabelle `jobs`. Jeder Job besitzt zunächst einen von der Datenbank erzeugten Primärschlüssel, der zur internen Identifikation dient. Zusätzlich wird für jeden Job ein vom Client bereitgestellter Idempotenzschlüssel gespeichert. Dieser stellt die in Z3 geforderte Idempotenz in der Joberstellung sicher, auf die in Kapitel 5.1.4 näher eingegangen wird. Um das zu gewährleisten, müssen die Idempotenzschlüssel eindeutig sein. Daher ist die Spalte `idempotency_key` mit einem Unique-Constraint zu belegen.

Neben den Identifikationsmerkmalen wird der eigentliche Inhalt des Jobs als Payload persistent gespeichert. Im Beispielsystem wird dieses Feld als JSON-String modelliert. Abhängig von der Art des Jobs und den Anforderungen an das System kann dies jedoch auch in jeglicher anderen Form umgesetzt werden, sofern die nötigen Informationen für die Durchführung des Jobs enthalten sind. Durch die Persistierung der Payload bleibt die vollständige Anfrage dauerhaft verfügbar und kann im Fehlerfall erneut verarbeitet werden, ohne dass der Client den Job nochmals übertragen muss. Das bildet die Grundlage für Z1 und Z9.

Zur Umsetzung des in Kapitel 5.1.2 vorgestellten Zustandsmodells besitzt jeder Job ein Statusattribut, welches den aktuellen Bearbeitungszustand beschreibt. Dieser Status wird für die Umsetzung von Z4-Z6 benötigt. Ergänzend hierzu werden weitere Metadaten gespeichert, die für die Umsetzung der Fehlertoleranz erforderlich sind. Das Attribut `created_at` speichert den Zeitpunkt der Joberstellung. Da Worker neue Jobs grundsätzlich in der Reihenfolge ihres Erstellungszeitpunkts beanspruchen, wird verhindert, dass ältere Jobs dauerhaft von neu eingehenden Jobs verdrängt werden und somit im Job-Store verbleiben. Dieses Verhalten adressiert Z10. Während der Bearbeitung wird im Attribut `claimed_by` gespeichert, welcher Worker einen Job aktuell verarbeitet. Diese Zuordnung stellt sicher, dass ausschließlich der Worker, der einen Job erfolgreich beansprucht hat, dessen Verarbeitung abschließen und das Ergebnis persistent speichern darf. Dadurch wird die in Z8 geforderte idempotente Ergebnisspeicherung unterstützt und verhindert, dass konkurrierende Worker im Recovery-Fall denselben Job gleichzeitig abschließen.

Für die Recovery-Strategie wird zusätzlich die Anzahl bisheriger Bearbeitungsversuche im Attribut `attempt_count` gespeichert. Dadurch kann die Anzahl automatischer Wiederholungsversuche begrenzt werden, sodass fehlerhafte Jobs nicht unbegrenzt erneut ausgeführt werden. Das Attribut `lease_until` dient der Umsetzung zeitbasierter Leases. Es gibt an, bis zu welchem Zeitpunkt ein Worker den beanspruchten Job exklusiv bearbeiten darf. Überschreitet die Bearbeitung diese Zeitspanne, kann der Recovery-Service den Job als verwaist erkennen und ihn entsprechend der definierten Recovery-Strategie erneut zur Bearbeitung freigeben. Diese beiden Attribute werden für die Umsetzung von Z9 und Z10 benötigt.

Die Verarbeitungsergebnisse werden in einer separaten Tabelle `job_result` gespeichert. Zwischen einem Job und seinem Ergebnis besteht dabei eine optionale Eins-zu-eins-Beziehung. Ein Job besitzt höchstens ein Ergebnis, da die idempotente Verarbeitung sicherstellt, dass erfolgreiche Jobs nur einmal abgeschlossen werden können. Gleichzeitig gehört jedes gespeicherte Ergebnis eindeutig zu genau einem Job. Solange ein Job noch nicht erfolgreich bearbeitet wurde, existiert dementsprechend kein zugehöriger Ergebniseintrag.

Das dargestellte Datenbankschema bildet das minimale Schema zur Umsetzung der in dieser Arbeit betrachteten Fehlertoleranzmechanismen. In realen Anwendungen kann der Job-Store beliebig um weitere fachliche Attribute oder zusätzliche Tabellen erweitert werden. Ebenso ist es möglich, eine bereits bestehende relationale Datenbank zur Umsetzung des Job-Processing-Systems zu verwenden, indem das vorgestellte Schema in die vorhandene Datenbank integriert wird.

5.1.4 Joberstellung

Die Annahme neuer Jobs gehört zur Erstellungsphase neuer Jobs und bildet somit den Einstiegspunkt in das entwickelte Job-Processing-System. Zur Umsetzung der Exactly-Once Auslieferung inklusive der Zusicherungen Z1 bis Z3 sind verschiedene Mechanismen nötig, die im Folgenden vorgestellt werden. Der Ablauf der Jobannahme ist in Abbildung 5.4 dargestellt. Zur Erstellung eines neuen Jobs sendet der Client eine Anfrage über die REST-Schnittstelle an das Backend. Die Anfrage besteht aus zwei Bestandteilen: der eigentlichen Nutzlast (*Payload*), welche die durchzuführende Arbeit beschreibt, sowie einem vom Client erzeugten Idempotenzschlüssel. Dieser Schlüssel identifiziert einen Job eindeutig und ermöglicht es, mehrfach übertragene Anfragen desselben Jobs zuverlässig zu erkennen. Die Verwendung des Idempotenzschlüssels ist insbesondere für den Fall relevant, dass der Client eine Anfrage erneut sendet, obwohl der ursprüngliche Job bereits erfolgreich gespeichert wurde. Ein solches Szenario kann beispielsweise auftreten, wenn das Acknowledgement des Backends bei der Übertragung verloren geht und der Client deshalb fälschlicherweise von einer fehlgeschlagenen Joberstellung ausgeht. Durch den Idempotenzschlüssel kann das Backend erkennen, dass der betreffende Job bereits existiert und eine doppelte Persistierung verhindern. Damit wird Zusicherung Z3 umgesetzt.

Nach Eingang der Anfrage leitet die REST-API diese an den Job-Service weiter. Dieser überprüft zunächst, ob bereits ein Job mit dem übermittelten Idempotenzschlüssel existiert. Ist dies nicht der Fall, wird der neue Job persistent im Job-Store gespeichert.

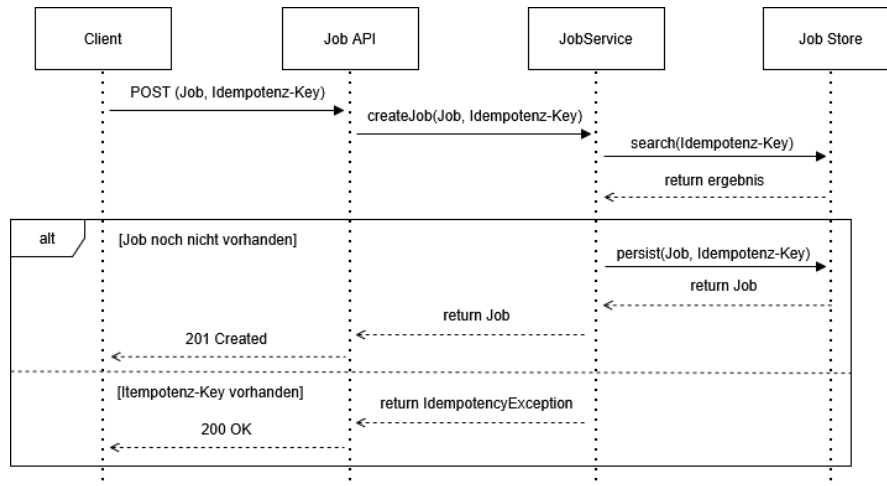


Abbildung 5.4: Ablauf der Jobannahme

Anschließend gibt der Service den erzeugten Job an die REST-API zurück, welche dem Client den HTTP-Statuscode **201 Created** als Bestätigung der erfolgreichen Joberstellung liefert.

Existiert dagegen bereits ein Job mit demselben Idempotenzschlüssel, wird keine erneute Persistierung durchgeführt. Stattdessen wirft der Service einen Fehler, der von der REST-API abgefangen wird. Da sich der gewünschte Job bereits im System befindet und somit der gewünschte Zielzustand erreicht wurde, beantwortet die API die Anfrage mit dem HTTP-Statuscode **200 OK**.

Kann während der Persistierung keine Verbindung zur Datenbank hergestellt werden oder tritt ein anderer transienter Fehler auf, wird der Persistierungsvorgang automatisch mehrfach mit exponentiellem Backoff wiederholt. Weitere Details zu Client- und Serverseitigen Retries werden in Kapitel 5.1.6 behandelt. Erst wenn der Job erfolgreich gespeichert wurde oder bereits existiert, wird dem Client ein erfolgreicher HTTP-Statuscode zurückgegeben. Kann der Job dagegen auch nach Ausschöpfen aller Wiederholungsversuche nicht gespeichert werden, antwortet das Backend mit dem HTTP-Statuscode **500 Internal Server Error**. Dadurch kann der Client erkennen, dass kein erfolgreicher Abschluss der Operation garantiert werden konnte und den Erstellungsversuch seinerseits gemäß der definierten Retry-Strategie wiederholen. Dieses Zusammenspiel zwischen client- und serverseitigen Wiederholungsmechanismen stellt sicher, dass Zusicherung Z1 und Z2 eingehalten werden und gleichzeitig kein direktes menschliches Eingreifen nach dem ersten fehlgeschlagenen Versuch notwendig ist.

5.1.5 Exklusive Verarbeitung und Transaktionsmodell

Eine naheliegende Möglichkeit zur Absicherung der Hintergrundverarbeitung vor konkurrierenden Zugriffen besteht darin, eine Datenbanktransaktion über den gesamten Lebenszyklus eines Jobs geöffnet zu halten. Die Transaktion würde unmittelbar nach dem

Beanspruchen eines Jobs beginnen und erst nach Abschluss der vollständigen Fachlogik sowie der Persistierung des Ergebnisses beendet werden. Wie bereits in Kapitel 2.2.2 erläutert, bringen lange Transaktionen einige Nachteile mit sich und daher ist dieser Ansatz auch für das in dieser Arbeit entworfene System ungeeignet. Transaktionen sollten so lang wie nötig und so kurz wie möglich entworfen werden. Hintergrundjobs kapseln typischerweise langlaufende Berechnungen oder E/A-intensive Operationen (Microsoft Corporation (Hrsg.) 2026a). Während der gesamten Bearbeitungsdauer würden dadurch Datenbankverbindungen sowie Sperren unnötig lange gehalten werden müssen.

Aus diesem Grund verfolgt das entwickelte System einen anderen Ansatz. Die eigentliche Fachlogik wird vollständig außerhalb einer Datenbanktransaktion ausgeführt. Datenbanktransaktionen werden ausschließlich zur atomaren Synchronisation der Zustandsübergänge verwendet und deshalb möglichst kurz gehalten. Die Kombination aus dem in Kapitel 5.1.2 vorgestellten Zustandsmodell und kurzlaufenden Transaktionen ermöglicht eine exklusive Bearbeitung der Jobs, ohne Datenbankressourcen während der eigentlichen Verarbeitung zu blockieren.

Der Verarbeitungsablauf gliedert sich hierzu in drei Phasen, die in Abbildung 5.5 visualisiert sind. Zunächst beansprucht ein Worker innerhalb einer Datenbanktransaktion

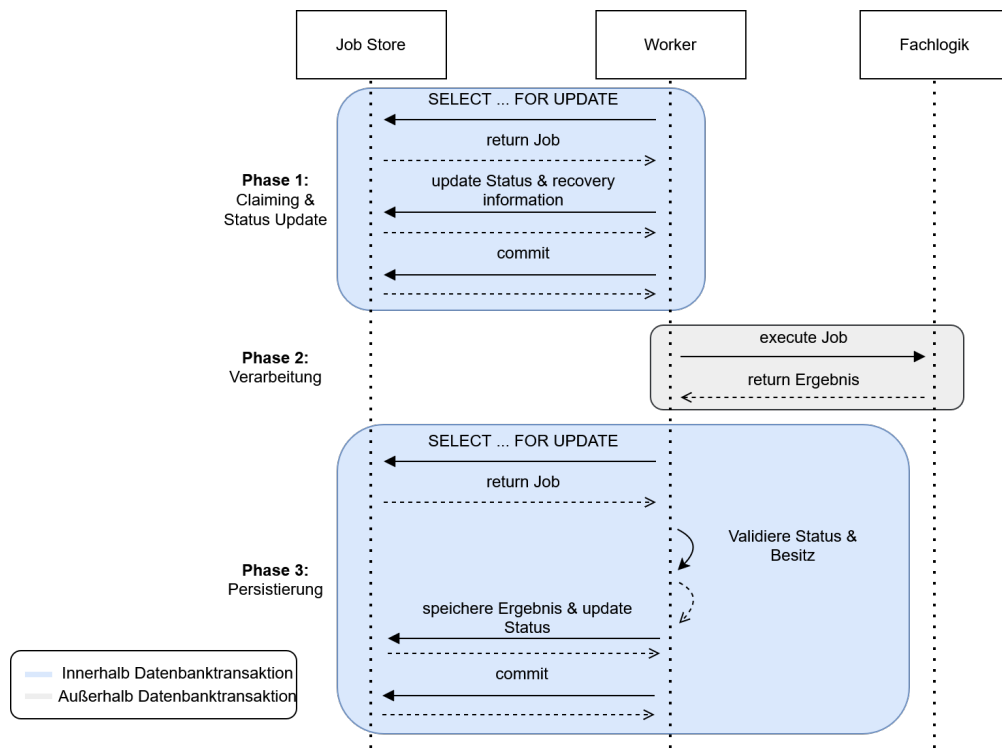


Abbildung 5.5: Transaktionsmodell

einen freien Job. Hierzu wird mittels `SELECT ... FOR UPDATE` ein geeigneter Job ausgewählt und exklusiv gesperrt. Zusätzlich wird dessen Status von *Queued* nach *Run-*

ning überführt und die Attribute für die Recovery wie `setClaimed_by`, `attempt_count` oder `lease_until` aktualisiert. Unmittelbar nach diesen Änderungen wird die Transaktion committet und die Datenbankverbindung wieder freigegeben. Anschließend erfolgt die eigentliche Verarbeitung des Jobs vollständig außerhalb einer Datenbanktransaktion. Da der Job bereits den Zustand *Running* besitzt, können andere Worker ihn währenddessen nicht erneut beanspruchen. Nach erfolgreichem Abschluss der eigentlichen Fachlogik darf ein Worker den Job nicht unmittelbar als erfolgreich markieren. Stattdessen wird zunächst eine neue Transaktion geöffnet, innerhalb der sowohl das Verarbeitungsergebnis persistent gespeichert als auch der Jobstatus atomar von *Running* nach *Successful* aktualisiert wird. Beide Änderungen werden gemeinsam committet. Muss ein Job dagegen aufgrund eines Fehlers oder eines abgelaufenen Leases zurückgesetzt werden, erfolgt auch dieser Zustandsübergang auf identische Weise. Dadurch ist sichergestellt, dass niemals ein Zustand entstehen kann, in dem zwar bereits ein Ergebnis gespeichert wurde, der Job jedoch weiterhin als *Running* oder *Queued* gekennzeichnet ist oder umgekehrt. Die gemeinsame Speicherung von Ergebnis und Jobstatus innerhalb derselben lokalen ACID-Transaktion gewährleistet somit die in Z7 geforderte Atomarität. Damit werden sämtliche Änderungen am Zustand eines Jobs atomar durchgeführt, während die eigentliche Fachlogik vollständig außerhalb der Transaktionsgrenzen verbleibt.

Für die Synchronisation konkurrierender Worker wird ein pessimistisches Sperrverfahren verwendet. Wie bereits in Kapitel 2.2.3 erläutert, eignen sich optimistische Synchronisationsverfahren insbesondere für Systeme, in denen konkurrierende Änderungen nur selten auftreten. Im entwickelten Job-Processing-System ist jedoch das Gegenteil der Fall. Da alle Worker neue Jobs entsprechend ihres Erstellungszeitpunkts aus derselben Warteschlange entnehmen, konkurrieren sie regelmäßig um dieselben Datensätze. Optimistische Verfahren würden in diesem Szenario zu einer hohen Anzahl abgebrochener Transaktionen und entsprechender Wiederholungsversuche führen. Kleppmann (2017, Kap. 7, Abschnitt „Pessimistic versus optimistic concurrency control“) weist darauf hin, dass sich optimistische Verfahren bei hoher Konkurrenz negativ auf den Gesamtdurchsatz auswirken, da die zusätzlich notwendigen Wiederholungen die Datenbank weiter belasten. Aus diesem Grund wird im Rahmen dieser Arbeit ein pessimistisches Sperrverfahren verwendet, bei dem konkurrierende Zugriffe bereits während der Auswahl eines Jobs verhindert werden. Dadurch kann sichergestellt werden, dass jeder Job zu jedem Zeitpunkt höchstens einem Worker exklusiv zugeordnet ist.

Ein weiterer Vorteil der gewählten Architektur ergibt sich daraus, dass sowohl der Job-Store als auch die Verarbeitungsergebnisse innerhalb derselben relationalen Datenbank gespeichert werden. Sämtliche Zustandsänderungen sowie die Persistierung eines Verarbeitungsergebnisses können dadurch innerhalb lokaler ACID-Transaktionen erfolgen. Die Koordination mehrerer unabhängiger Systeme und damit die Notwendigkeit verteilter Transaktionen entfallen vollständig.

5.1.6 Retry-Strategie

Auf Grundlage der in Kapitel 2.3.2 beschriebenen Prinzipien werden Wiederholungsmechanismen gezielt an denjenigen Stellen der Architektur eingesetzt, an denen temporäre

Infrastrukturfehler auftreten können, ohne dass die eigentliche Fachlogik fehlerhaft ist. Ziel ist es, kurzfristige Störungen bereits während der Verarbeitung zu kompensieren und dadurch unnötige Recovery-Vorgänge oder Benutzerinteraktionen zu vermeiden.

Hierzu unterscheidet die Architektur zwischen der Kommunikation des Clients mit dem Backend und den internen Datenbankzugriffen der Worker und der Job-API. Beide Kommunikationswege stellen voneinander unabhängige Fehlerquellen dar und können daher unabhängig voneinander von transienten Infrastrukturfehlern betroffen sein.

Auf Clientseite werden Wiederholungsversuche für die Kommunikation mit der REST-Schnittstelle des Backends vorgesehen. Kann eine Anfrage aufgrund einer temporären Netzwerkstörung oder einer kurzfristigen Nichtverfügbarkeit des Backends nicht erfolgreich abgeschlossen werden, wird die Anfrage automatisch erneut übertragen. Da die Joberstellung durch den in Kapitel 5.1.4 beschriebenen Idempotenzmechanismus abgesichert ist, führt eine erneute Übertragung derselben Anfrage nicht zu einer mehrfachen Persistierung eines Jobs. Dadurch kann insbesondere der Fall behandelt werden, dass der Job bereits erfolgreich gespeichert wurde, die Bestätigung über den erfolgreichen Abschluss den Client jedoch aufgrund eines Kommunikationsfehlers nicht erreicht. Die Kombination aus Retry und Idempotenz schafft somit eine Exactly-Once-Semantik für die Joberstellung unter der Voraussetzung, dass der Fehler tatsächlich transient ist und die Parameter für Retry und Backoff passend konfiguriert sind.

Innerhalb des Backends werden Wiederholungsversuche ausschließlich für Datenbankoperationen vorgesehen, deren Fehlschlagen typischerweise auf transiente Infrastrukturfehler zurückzuführen ist, beispielsweise kurzzeitige Netzwerkunterbrechungen oder die vorübergehende Nichtverfügbarkeit der Datenbank. Dies betrifft insbesondere das Reservieren eines Jobs sowie das Persistieren eines Verarbeitungsergebnisses. Kann eine dieser Operationen aufgrund einer vorübergehend nicht erreichbaren Datenbank oder einer vergleichbaren transienten Störung nicht abgeschlossen werden, wird sie automatisch erneut ausgeführt. Dagegen werden fachliche Fehler während der eigentlichen Jobverarbeitung bewusst nicht wiederholt, da deren Ursache in der Regel nicht durch einen erneuten Ausführungsversuch beseitigt werden kann.

Zur strukturellen Entkopplung der Wiederholungslogik wird diese in einem eigenständigen Retry-Teilsystem gekapselt. Wie in Abbildung 5.6 dargestellt, nutzen die jeweiligen Komponenten, wie beispielsweise der Client im Frontend zur Erstellung der Jobs, einen zentralen RetryExecutor, der die auszuführende Operation über die Methode *execute(...)* entgegennimmt und deren Ausführung entsprechend einer konfigurierbaren Retry-Policy steuert. Der RetryExecutor enthält dabei selbst keine anwendungsspezifischen Entscheidungen darüber, wann eine Wiederholung erfolgen soll. Stattdessen fängt er Exceptions von der auszuführenden Operation und gibt sie an *shouldRetry(...)* weiter. Die spezifische Policy entscheidet dann, ob ein Retry sinnvoll ist oder nicht, indem sie die Exception klassifiziert und einen entsprechenden Wahrheitswert zurückliefert. Die Retry-Policies implementieren dabei eigene unterschiedliche Kriterien für Wiederholungsversuche, die sich in den behandelten Fehlerarten unterscheiden, sowie das zugehörige Backoff-Verhalten. Dadurch kann dasselbe Retry-Teilsystem sowohl für die Kommunikation zwischen Client und Backend als auch für interne Datenbankzugriffe der Worker verwendet werden, obwohl sich die jeweils behandelten Fehlerarten unterscheiden. Die Retry-Logik bleibt

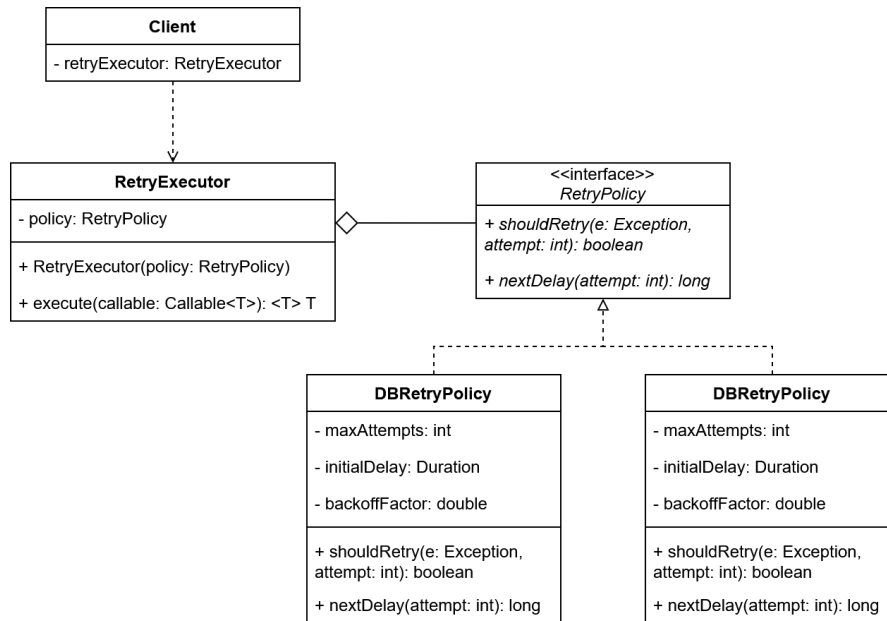


Abbildung 5.6: Klassendiagramm der Retry-Komponente

dadurch zentral gekapselt, während die konkrete Fehlerbehandlung an den jeweiligen Anwendungskontext angepasst werden kann. Die konkrete Implementierung des **Retry-Executors** sowie die Konfiguration der einzelnen **Retry-Policies** werden in Kapitel 5.2 erläutert.

5.1.7 Recovery

Die in Kapitel 5.1.6 beschriebene **Retry-Strategie** behandelt ausschließlich transiente Infrastrukturfehler, bei denen die ausführende Komponente weiterhin aktiv ist. Fällt ein Worker dagegen während der Verarbeitung vollständig aus oder bleibt dauerhaft hängen, kann kein lokaler **Retry** mehr erfolgen. Zur Behandlung solcher Fehler wird die Architektur daher um einen eigenständigen **Recovery-Service** ergänzt.

Wie in Abbildung 5.7 dargestellt, überprüft der **Recovery-Service** den **Job-Store** in regelmäßigen Abständen auf potenziell verwaiste Jobs. Hierzu werden sämtliche Jobs betrachtet, die sich weiterhin im Zustand *Running* befinden, deren *Lease* jedoch bereits abgelaufen ist. Ein abgelaufener *Lease* wird dabei als Indikator dafür verwendet, dass die ursprüngliche Verarbeitung nicht innerhalb der vorgesehenen Zeit abgeschlossen werden konnte. Für jeden erkannten verwaisten Job entscheidet der **Recovery-Service** anschließend über das weitere Vorgehen. Wurde die maximal zulässige Anzahl an Ausführungsversuchen noch nicht erreicht, wird der Job erneut zur Verarbeitung freigegeben. Hierzu werden innerhalb einer zusammengehörigen Transaktion der Status auf *Queued* zurückgesetzt sowie die Attribute *claimed_by* und *lease_until* zurückgesetzt. Anschließend kann der Job von einem beliebigen Worker erneut beansprucht werden. Wurde die

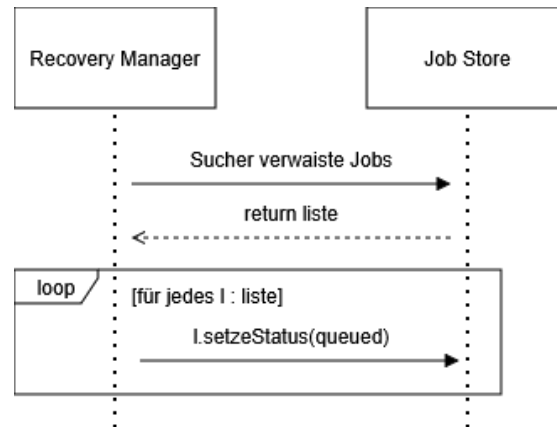


Abbildung 5.7: Ablauf des Recovery-Mechanismus

konfigurierte Obergrenze der Wiederholungsversuche bereits erreicht, wird der Job stattdessen in den Endzustand *Failed* überführt und nicht erneut verarbeitet. Dadurch wird verhindert, dass dauerhaft fehlerhafte Jobs unbegrenzt Systemressourcen belegen.

Durch diesen Mechanismus können insbesondere Worker-Abstürze, JVM-Abstürze, Container-Neustarts sowie dauerhaft blockierte Worker in Bezug auf die Jobausführung behandelt werden. Die eigentliche Terminierung oder der Neustart fehlerhafter Worker ist dagegen nicht Bestandteil des entwickelten Systems. Diese Aufgabe wird bewusst an eine externe Prozessüberwachung oder Orchestrierungsplattform, beispielsweise Kubernetes, delegiert. Das entwickelte System stellt lediglich sicher, dass durch den Ausfall eines Workers kein Job dauerhaft verloren geht.

Die Wiederaufnahme eines Jobs erfolgt in der entwickelten Architektur grundsätzlich nicht durch das Fortsetzen eines unterbrochenen Ausführungszustands, sondern durch eine vollständige Neuausführung auf Basis der ursprünglich persistierten Eingabedaten. Aus diesem Grund muss die Payload eines Jobs mindestens bis zum erfolgreichen Abschluss unverändert im Job-Store verbleiben. Dadurch kann jeder Wiederholungsversuch unter identischen Ausgangsbedingungen beginnen. In Kombination mit der atomaren Speicherung des Verarbeitungsergebnisses wird sichergestellt, dass unvollständig abgeschlossene Ausführungsversuche keine inkonsistenten Zustände hinterlassen und den Erfolg späterer Wiederholungen nicht beeinflussen.

Problematisch ist ein solches Vorgehen jedoch, wenn während der Verarbeitung externe Seiteneffekte entstehen. Wird beispielsweise im Rahmen eines Jobs eine E-Mail versendet und der Worker stürzt anschließend vor dem erfolgreichen Abschluss der Verarbeitung ab, so wird der Job durch den Recovery-Service erneut zur Verarbeitung freigegeben. Die erneute Ausführung würde in diesem Fall dieselbe E-Mail ein weiteres Mal versenden.

Die entwickelte Infrastruktur garantiert daher ausschließlich die idempotente Verarbeitung innerhalb des Job-Processing-Systems, hinsichtlich der Reservierung von Jobs und der atomaren Speicherung von Verarbeitungsergebnissen. Für externe Seiteneffekte kann eine Exactly-Once-Ausführung dagegen grundsätzlich nicht gewährleistet werden.

Diese ist nur möglich, wenn das jeweils aufgerufene Zielsystem selbst eine idempotente Verarbeitung unterstützt, beispielsweise durch Nutzung von Idempotenzschlüsseln wie im Beispielsystem bei der Joberstellung (vgl. Kapitel 5.1.4). Unterstützt das Zielsystem keine entsprechende Semantik, kann eine Mehrfachausführung von Seiteneffekten infolge eines Recovery-Vorgangs nicht ausgeschlossen werden.

Ein Nebeneffekt dieser Architektur besteht darin, dass ein Job zeitweise von mehreren Workern gleichzeitig bearbeitet werden kann. Überschreitet beispielsweise ein Worker seine Lease-Dauer, wird der Job erneut freigegeben, obwohl der ursprüngliche Worker seine Verarbeitung möglicherweise noch beendet. Die beschriebene Recovery-Strategie versucht damit, die Erfolgswahrscheinlichkeit eines Jobs zu erhöhen. Sie verfolgt damit das Ziel einer At-least-Once-Ausführung von Jobs. Die daraus resultierenden Anforderungen an die idempotente Verarbeitung werden im folgenden Abschnitt betrachtet.

5.1.8 Idempotente Ergebnisgenerierung

Z8 verlangt, dass pro Job maximal ein Ergebnis erzeugt wird. Das in Kapitel 5.1.5 vorgestellte Claiming-Verfahren stellt sicher, dass ein Job im Normalfall ausschließlich von einem einzelnen Worker bearbeitet wird. Durch den in Kapitel 5.1.7 genauer betrachteten Recovery-Fall kann diese Eigenschaft jedoch nicht immer garantiert werden. Daher muss sichergestellt werden, dass das fachliche Ergebnis nur einmal gespeichert werden kann oder die Fachlogik auch bei mehrfacher Ausführung in jedem Fall immer das gleiche Ergebnis erzeugt. Da der Jobstatus sowie die Jobergebnisse in derselben Datenbank verwaltet werden, bietet sich die Umsetzung der ersten Option mithilfe von Transaktionen an. Um die idempotente Verarbeitung auf diese Weise sicherzustellen, darf ein Worker ein Verarbeitungsergebnis ausschließlich dann persistent speichern, wenn er laut Job-Store weiterhin der Besitzer des Jobs ist. Vor der Speicherung wird daher der betroffene Datensatz gesperrt und innerhalb einer transaktional geschützten Operation erneut der aktuelle Jobstatus, die Worker-ID sowie die Gültigkeit des Lease überprüft. Nur wenn der Job sich weiterhin im Zustand *Running* befindet, die gespeicherte Worker-ID mit der des ausführenden Workers übereinstimmt und das Lease noch nicht abgelaufen ist, wird das Ergebnis übernommen und der Job erfolgreich abgeschlossen. Andernfalls wird der Speichervorgang verworfen, da bereits ein anderer Worker die weitere Verarbeitung übernommen hat oder der Job bereits abgeschlossen wurde. Dadurch kann jeder Job höchstens einmal erfolgreich gespeichert werden, selbst wenn mehrere Worker denselben Job parallel ausgeführt haben. Mit diesem Vorgehen wird jedoch nicht das in Kapitel 5.1.7 erwähnte Problem verhindert, dass Seiteneffekte in externen Systemen ebenfalls idempotent sind.

Pro Kapitel eine solche Tabelle um zu erklären welche Architekturentscheidung genau welche Zusicherung erfüllt

5.2 Implementierung

(Hohpe und Woolf 2005, Kap. 5 Abschnitt „Command Message“) (Richardson 2018, Kap. 6.2)

Problem	Betroffene Zusicherungen
Doppelte Requests	SZ1 SZ3
Mehrere Worker	SZ6
Recovery	SZ9 SZ10

Tabelle 5.1:

Zusicherung	DB-Unterstützung
Job geht nicht verloren	persistente Job-Tabelle
Keine doppelten Jobs	UNIQUE(idempotency_key)
Genau ein Zustand	status ENUM
Gültige Übergänge	Anwendung + Transaktionen
Exklusive Verarbeitung	claimed_by + lease_until + atomares Claiming
Kein zweites Ergebnis	1:1-Beziehung Job <-> JobResult
Recovery	lease_until
Retry	retry_count + next_retry_at

Tabelle 5.2:

5.2.1 Technologie-Stack

5.2.2 Zentrale Code-Entscheidungen

Begründen, warum ich mich für diese Frameworks entschieden habe, die vorkommen. -> Jede (auch implizite) Entscheidung begründen Alles Begründen!!! Wieso ist das überhaupt relevant? Welches Problem löse ich, wieso habe ich das so gemacht? Wieso genau diese Fehlerszenarien? Begründen wieso nicht andere oder mehr? Gibt es formale Modelle die die Zusicherungen zu jedem Zeitpunkt checken?

6 Evaluation

6.1 Fehlerszenarien

6.2 Durchführung

6.3 Ergebnisse

7 Diskussion

7.1 Gewonnene Erkenntnisse

7.2 Einschränkungen und Grenzen der Arbeit

Diese Arbeit betrachtet die Fehlertoleranz ab dem Zeitpunkt, zu dem ein Job persistent im Job-Store vorliegt; die zuverlässige Auftragserzeugung/Client-Kommunikation (z.B. Outbox, Messaging) ist nicht Gegenstand. Consumers may crash at any time, so it could happen that a broker delivers a message to a consumer but the consumer never processes it, or only partially processes it before crashing. In order to ensure that the message is not lost, message brokers use *acknowledgments*: a client must explicitly tell the broker when it has finished processing a message so that the broker can remove it from the queue." (Kleppmann 2017, Kap.11 -> Acknowledgments and redelivery bzw. Message brokers)

REST hat keinen Puffer und ist synchron -> Client und Server muss während der ganzen Kommunikation laufen, damit Kommunikation funktioniert. Weitere Nachteile in Referenc: (Richardson 2018, Kap.3.2.2)

Kein Fokus auf Kommunikation im allgemeinen, weder wie oben genannt die Erstellung von Jobs, als auch die Benachrichtigung bei Fertigstellung. Dafür bräuchte es Transaktionales messaging bspw. mittels transactional outbox pattern (Richardson 2018, Kap.3.3.7) oder asynchrones Messaging (Richardson 2018, Kap.3.4.2)

Es gibt keine perfekte reliability (Kleppmann 2017, Kap. 1, Abschnitt „Reliability“)

Load Leveling umgesetzt?

Performance von DB gegenüber Message Queues

Keine Fehlertoleranz in Bezug auf Datenbeständigkeit (z.B. Festplatte des Job-Stores geht kaputt), Naturkatastrophen (Ganzer Standort fällt aus z.B. wegen Überschwemmung) (Newman 2026, Kap. 9)

Es gibt keine Möglichkeit absolute Garantien zu geben, nur viele Möglichkeiten der Risikoreduktion (Kleppmann 2017, Kap. 7 Abschnitt „The Meaning of ACID“-> Ende)

um performance zu verbessern könnte man verschiedene checkpoints in die verarbeitung einbringen, sodass bei einem Abbrechen einer Task nicht die gesamte Task neu gestartet wird sondern ab einem bestimmten punkt wieder angefangen wird (Microsoft Corporation (Hrsg.) 2026a, Abschnitt „Reliability considerations“).

vorsicht bei dingen bei denen kein rollback gemacht werden kann: versendet ein worker eine email, stürzt ab und die operation wird wiederholt wird potenziell noch eine email versendet. das muss gesondert abgefangen werden

Es können Probleme entstehen, wenn zu viele Anfragen rein kommen, die die DB nicht gleichzeitig verarbeiten kann (vgl. Exmaple (Microsoft Corporation (Hrsg.) 2026b)) Dafür bräuchte es separates Queue-Based Load Leveling für das Messaging an sich, das System konzentriert sich auf Queue-Based Load Leveling bzgl der Jobverarbeitung. Es werden zwar Clientseitige Retries durchgeführt, das ist aber keine Garantie, dass die Nachricht trotz hoher Last ankommt

bietet sich vor allem an, wenn man sowieso schon eine db nutzt, man braucht keine zusätzliche Infrastruktur (im Vergleich zum Message Broker, der als eigener Server läuft)

Bei sehr großen Systemen kann die Datenbank aufgrund einer hohen Anzahl Zugriffsversuche jedoch zum limitierenden Faktor werden, sodass dedizierte Messaging-Systeme hinsichtlich des erreichbaren Durchsatzes Vorteile bieten (Kleppmann 2017, Kap. 11, Abschnitt „Rebuilding state after a failure“).

8 Fazit und Ausblick

Literaturverzeichnis

- Apache Software Foundation (Hrsg.) (22. Mai 2026). *Apache Kafka Documentation 4.3.X*. AK 4.3.X. URL: <https://kafka.apache.org/43/> (besucht am 26.06.2026).
- Bass, Len (Aug. 2021). *Software architecture in practice*. Unter Mitarb. von Paul Clements und Rick Kazman. Fourth edition. SEI series in software engineering. Boston: Addison-Wesley.
- Broadcom Inc. (Hrsg.) (15. Juni 2026a). *RabbitMQ Documentation / RabbitMQ*. URL: <https://www.rabbitmq.com/docs> (besucht am 26.06.2026).
- (24. Mai 2026b). *Spring Boot*. Spring Boot. URL: <https://spring.io/projects/spring-boot> (besucht am 24.05.2026).
- Hangfire OÜ (Hrsg.) (26. Juni 2026). *Hangfire – Background jobs and workers for .NET and .NET Core*. URL: <https://www.hangfire.io> (besucht am 26.06.2026).
- Hohpe, Gregor und Bobby Woolf (2005). *Enterprise integration patterns designing, building, and deploying messaging solutions*. 5. print. Boston [u.a.: Addison-Wesley.
- Kleppmann, Martin (2017). *Designing data-intensive applications: the big ideas behind reliable, scalable, and maintainable systems*. First edition. Beijing, [China: O'Reilly.
- Kudraß, Thomas (5. März 2015). *Taschenbuch Datenbanken*. Hrsg. von Kudraß Thomas. Carl Hanser Verlag GmbH & Co. KG. ISBN: 978-3-446-43508-7. DOI: 10.3139/9783446440265.fm. URL: <https://www.hanser-elibrary.com/doi/10.3139/9783446440265.fm> (besucht am 21.06.2026).
- Microsoft Corporation (Hrsg.) (31. März 2026a). *Best Practices for Background Jobs - Azure Architecture Center*. URL: <https://learn.microsoft.com/en-us/azure/architecture/best-practices/background-jobs> (besucht am 17.06.2026).
- (12. Juni 2026b). *Queue-Based Load Leveling Pattern - Azure Architecture Center*. URL: <https://learn.microsoft.com/en-us/azure/architecture/patterns/queue-based-load-leveling> (besucht am 17.06.2026).
- Newman, Sam (Sep. 2026). *Building Resilient Distributed Systems*. O'Reilly. URL: <https://learning.oreilly.com/library/view/building-resilient-distributed/9781098163532/> (besucht am 24.05.2026).
- Richardson, Chris (2018). *Microservices patterns*. Software development. Shelter Island: Manning.
- Rosoco BV (Hrsg.) (24. Mai 2026). *JobRunr Documentation*. URL: <https://www.jobrunr.io/en/documentation/> (besucht am 24.05.2026).
- Saake, Gunter, Andreas Heuer und Kai-Uwe Sattler (Juni 2018). *Datenbanken – Konzepte und Sprachen*. mitp Verlags GmbH & Co KG. ISBN: 978-3-95845-778-2. URL: <https://learning.oreilly.com/library/view/datenbanken-konzepte/9783958457782/> (besucht am 19.06.2026).
- Temporal Technologies Inc. (24. Mai 2026). *Temporal Docs*. URL: <https://docs.temporal.io/> (besucht am 24.05.2026).
- Terracotta Inc. (Hrsg.) (24. Mai 2026). *Quartz Scheduler Documentation*. URL: <https://www.quartz-scheduler.org/documentation/quartz-2.5.x/> (besucht am 24.05.2026).

The PostgreSQL Global Development Group (Hrsg.) (14. Mai 2026). *PostgreSQL 18.4 Documentation*. PostgreSQL Documentation. URL: <https://www.postgresql.org/docs/18/index.html> (besucht am 21.06.2026).